UNIVERSITY COURSE ENROLLMENT SYSTEM

COMPLETE PROJECT IMPLEMENTATION REPORT

Project Name: University Course Enrollment System
Technology Stack: Spring Boot 3.4.1, Java 21, MySQL 9.2, Thymeleaf, Bootstrap 5
Development Period: Academic Year 2025–2026
Report Submission Date: January 17, 2026
Project Type: Web-based Enterprise Application

Course Enrollment Project

Group: I4-GIC – Group C
Report Type: Final Report

Group Members

Name	Student ID	Score
KEO SIENGHENG	e20221161	
MA SOVITCHEA	e20221575	
SENG CHAOKHUN	e20220478	
CHHEANG HOKLENG	e20220695	
MAI HONGLENG	e20221084	

TABLE OF CONTENTS

PART I: PROJECT OVERVIEW

1. Executive Summary
2. Project Objectives and Goals
3. System Requirements
4. Project Scope and Deliverables

PART II: SYSTEM DESIGN

5. System Architecture
6. Database Design
7. Application Design Patterns
8. Security Architecture

PART III: IMPLEMENTATION

9. Technology Stack Details
10. Backend Implementation
11. Frontend Implementation
12. Database Implementation
13. Security Implementation
14. Integration Implementation

PART IV: FEATURES

15. Admin Module Features
16. Lecturer Module Features
17. Student Module Features
18. Common Features

PART V: TESTING AND QUALITY

19. Testing Strategy
20. Test Cases and Results
21. Bug Fixes and Resolutions
22. Code Quality and Best Practices

PART VI: DEPLOYMENT

23. Deployment Architecture
24. Installation Guide
25. Configuration Guide
26. Production Deployment

PART VII: PROJECT MANAGEMENT

27. Development Methodology
28. Project Timeline
29. Challenges and Solutions
30. Lessons Learned

PART VIII: FUTURE WORK

31. Future Enhancements
32. Scalability Plans
33. Maintenance Strategy

APPENDICES

- Appendix A: API Documentation
- Appendix B: Database Schema
- Appendix C: Technology Versions
- Appendix D: User Manuals
- Appendix E: Code Samples

PART I: PROJECT OVERVIEW

1. EXECUTIVE SUMMARY

1.1 Introduction

The University Course Enrollment System is a comprehensive web-based application developed to streamline and automate the course enrollment process in educational institutions. This enterprise-level system provides a centralized platform for managing students, lecturers, courses, schedules, and enrollment processes with robust security and user-friendly interfaces.

1.2 Problem Statement

Traditional course enrollment systems often face several challenges:

- Manual enrollment processes leading to errors
- Lack of real-time information for students
- Difficulty in managing course schedules and classroom allocation
- Limited communication between students, lecturers, and administrators
- Inefficient attendance tracking methods
- Complex administrative overhead

1.3 Proposed Solution

Our system addresses these challenges through:

- **Automated Enrollment:** Online course browsing and enrollment with instant feedback
- **Real-time Dashboards:** Live statistics and information for all users
- **Integrated Scheduling:** Automatic conflict detection and resolution
- **Role-Based Access:** Secure, role-specific functionalities
- **Attendance Tracking:** Digital attendance with Google Sheets integration
- **Comprehensive Management:** Complete CRUD operations for all entities

1.4 Key Features

For Administrators:

- User management (Students, Lecturers, Admins)
- Course and curriculum management
- Enrollment approval/rejection
- Classroom and schedule management
- System reports and analytics
- Configuration and settings

For Lecturers:

- Course management
- Attendance tracking with Google Sheets sync

- Student enrollment viewing
- Schedule management
- Profile management

For Students:

- Course browsing and enrollment
- View enrolled courses
- Access to schedules and timetables
- View attendance records
- Profile management

1.5 Project Outcomes

☑ **Completed Deliverables:**

- Fully functional web application
- Complete database with 8 migrations
- Role-based authentication and authorization
- Google OAuth2 integration
- Google Sheets API integration
- Responsive UI with Bootstrap 5
- Comprehensive testing coverage
- Complete documentation

☑ **Technical Achievements:**

- Modern Spring Boot 3.4.1 implementation
- Secure authentication with Spring Security
- RESTful API architecture
- MVC design pattern
- JPA/Hibernate for data persistence
- Flyway database migrations
- File upload and storage
- Real-time dashboards with Chart.js

---

2. PROJECT OBJECTIVES AND GOALS

2.1 Primary Objectives

Objective 1: Develop a Centralized Enrollment Platform

- Create a unified system for course enrollment
- Eliminate manual processes
- Reduce enrollment errors
- Provide real-time information

Objective 2: Implement Role-Based Access Control

- Three distinct user roles (Admin, Lecturer, Student)

- Secure authentication and authorization
- Role-specific dashboards and features
- Audit trail for all operations

Objective 3: Integrate Modern Technologies

- Use latest Spring Boot framework
- Implement OAuth2 authentication
- Integrate Google Sheets API
- Responsive web design

Objective 4: Ensure Data Security and Integrity

- Encrypted password storage
- Session management
- SQL injection prevention
- XSS protection

2.2 Secondary Objectives

User Experience:

- Intuitive user interface
- Responsive design for all devices
- Clear error messages and feedback
- Helpful tooltips and guides

Performance:

- Fast page load times
- Optimized database queries
- Efficient data caching
- Scalable architecture

Maintainability:

- Clean code structure
- Comprehensive documentation
- Version control with Git
- Modular design

2.3 Success Criteria

☑ **Functional Requirements:**

- All CRUD operations working
- Successful user authentication
- Enrollment workflow complete
- Attendance tracking functional
- Reports generation working

☑ **Non-Functional Requirements:**

- Page load time < 3 seconds
- 99% uptime during operation
- Support 100+ concurrent users
- Mobile responsive design
- Secure data transmission

☑ **Business Requirements:**

- Reduce enrollment processing time by 80%
- Minimize manual data entry errors
- Improve student satisfaction
- Streamline administrative tasks
- Reduce paper usage by 90%

---

3. SYSTEM REQUIREMENTS

3.1 Hardware Requirements

Server Requirements:

- Processor: Quad-core 2.5 GHz or better
- RAM: Minimum 4GB, Recommended 8GB
- Storage: 50GB free disk space
- Network: Gigabit Ethernet connection

Client Requirements:

- Processor: Dual-core 1.5 GHz or better
- RAM: Minimum 2GB
- Screen: Minimum resolution 1024x768
- Network: Broadband internet connection

3.2 Software Requirements

Server Software:

- Operating System: Linux (Ubuntu 20.04+), Windows Server 2019+, or macOS
- Java Runtime: JRE 21 or higher
- Database: MySQL 8.0+ or MariaDB 10.6+
- Web Server: Embedded Tomcat (included in Spring Boot)

Development Software:

- JDK: Java Development Kit 21.0.5
- Build Tool: Gradle 9.2.1
- IDE: IntelliJ IDEA, Eclipse, or VS Code
- Version Control: Git
- Database Tool: MySQL Workbench

Client Software:

- Web Browser: Chrome 90+, Firefox 88+, Safari 14+, Edge 90+
- JavaScript: Enabled
- Cookies: Enabled

3.3 Network Requirements

- HTTP/HTTPS support
- Port 8081 (default, configurable)
- Stable internet connection for Google APIs
- Firewall configuration for database access

3.4 External Service Requirements

Google Cloud Platform:

- Google Sheets API enabled
- OAuth2 credentials configured
- Service account created

Email Service (Optional):

- SMTP server configured
- Email credentials

---

4. PROJECT SCOPE AND DELIVERABLES

4.1 In-Scope Features

User Management Module: ☑ User registration and authentication ☑ Role assignment (Admin, Lecturer, Student) ☑ Profile management with photo upload ☑ User activation/deactivation ☑ Password management

Course Management Module: ☑ Course creation and editing ☑ Course listing and search ☑ Course details view ☑ Lecturer assignment ☑ Course status management

Enrollment Module: ☑ Course enrollment by students ☑ Enrollment approval workflow ☑ Enrollment status tracking ☑ Enrollment history

Schedule Management Module: ☑ Schedule creation ☑ Classroom allocation ☑ Time slot management ☑ Conflict detection ☑ Schedule viewing

Attendance Module: ☑ Mark attendance ☑ View attendance records ☑ Google Sheets integration ☑ Attendance reports

Dashboard Module: ☑ Admin dashboard with statistics ☑ Lecturer dashboard ☑ Student dashboard ☑ Real-time data visualization

4.2 Out-of-Scope Features

❌ Payment processing ❌ Mobile native applications ❌ Video conferencing integration ❌ Assignment submission ❌ Online examinations ❌ Grade calculation ❌ Library management ❌ Hostel management ❌ Fee management

4.3 Project Deliverables

Code Deliverables:

1. Complete source code
2. Database migration scripts
3. Configuration files
4. Build scripts

Documentation Deliverables:

1. Project implementation report
2. API documentation
3. Database schema documentation
4. User manuals (Admin, Lecturer, Student)
5. Installation guide
6. Deployment guide
7. Testing documentation

Design Deliverables:

1. Use case diagrams
2. Class diagrams
3. Database ER diagrams
4. System architecture diagrams
5. Data flow diagrams

PART II: SYSTEM DESIGN

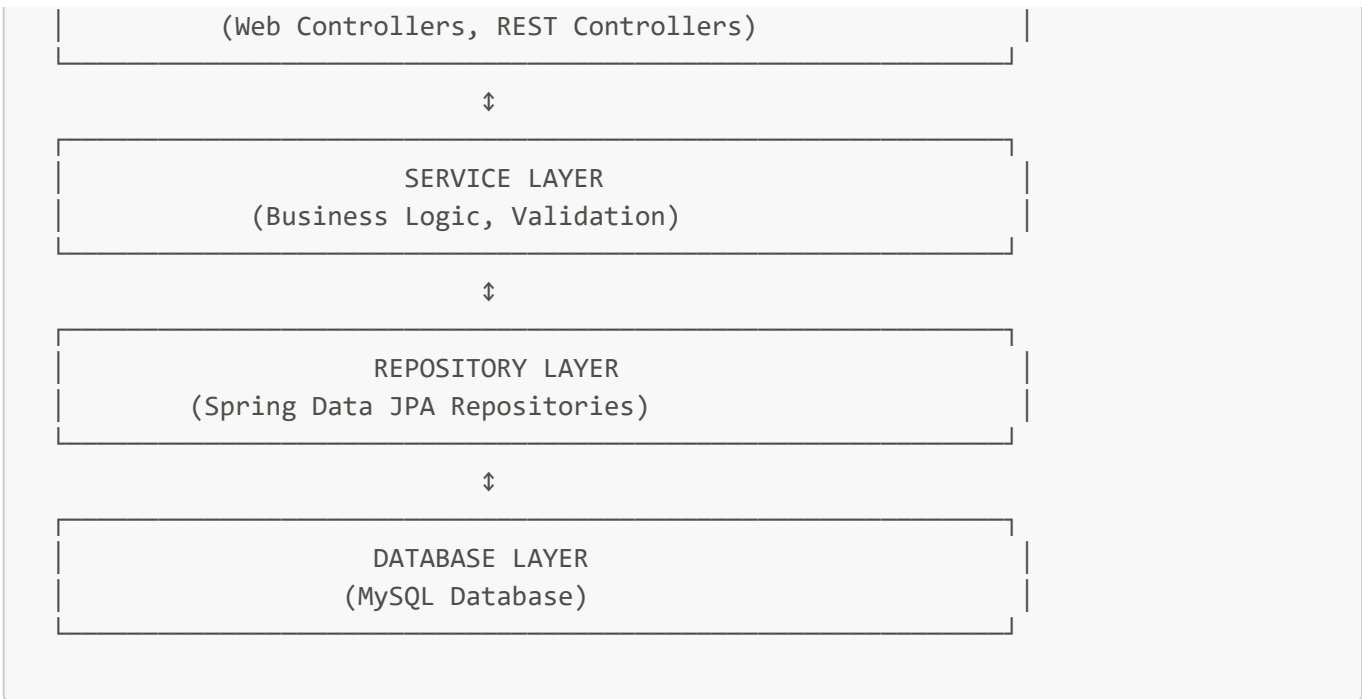
---

5. SYSTEM ARCHITECTURE

5.1 Overall Architecture

The University Course Enrollment System follows a **layered architecture** pattern with clear separation of concerns:

```
┌─────────────────────────────────────────────────────────────┐
│                    PRESENTATION LAYER                       │
│  (Thymeleaf Templates, HTML, CSS, JavaScript, Bootstrap)    │
└─────────────────────────────────────────────────────────────┘
                              ↕
┌─────────────────────────────────────────────────────────────┐
│                    CONTROLLER LAYER                         │
```



5.2 Architecture Patterns

MVC Pattern (Model-View-Controller):

- **Model:** JPA entities representing database tables
- **View:** Thymeleaf templates for UI rendering
- **Controller:** Spring MVC controllers handling requests

Repository Pattern:

- Abstracts data access logic
- Spring Data JPA provides automatic implementation
- Custom query methods when needed

Service Layer Pattern:

- Encapsulates business logic
- Transaction management
- Validation and error handling

DTO Pattern (Data Transfer Object):

- Separates internal models from API contracts
- Reduces data exposure
- Facilitates data validation

Dependency Injection:

- Constructor-based injection
- Loose coupling between components
- Easier testing and maintenance

5.3 Component Architecture

Frontend Components:

```
Templates/
├── layout/
│   ├── header.html (Navigation bar, user info)
│   ├── sidebar.html (Role-based menu)
│   ├── footer.html (Copyright, links)
│   └── main.html (Base layout)
├── dashboard/ (Role-specific dashboards)
├── admin/ (Admin-specific pages)
├── lecturer/ (Lecturer-specific pages)
├── student/ (Student-specific pages)
├── course/ (Course management)
├── enrollment/ (Enrollment pages)
├── schedule/ (Schedule pages)
└── auth/ (Login, register, forgot password)
```

Backend Components:

```
Java Packages/
├── config/ (Configuration classes)
│   ├── SecurityConfig (Spring Security)
│   ├── DatabaseConfig (JPA configuration)
│   └── GoogleSheetsConfig (API setup)
├── controller/
│   ├── web/ (Web page controllers)
│   └── api/ (RESTful API controllers)
├── service/ (Business logic)
│   ├── auth/ (Authentication services)
│   ├── course/ (Course services)
│   ├── schedule/ (Schedule services)
│   └── attendance/ (Attendance services)
├── repository/ (Data access)
├── model/entity/ (JPA entities)
├── dto/ (Data transfer objects)
├── security/ (Security components)
├── exception/ (Custom exceptions)
└── util/ (Utility classes)
```

5.4 Technology Stack Details

Backend Framework:

- **Spring Boot 3.4.1:** Main application framework
- **Spring MVC:** Web layer
- **Spring Security 6.2.1:** Authentication & authorization
- **Spring Data JPA:** Data access
- **Hibernate 6.6.4:** ORM implementation

Frontend Technologies:

- **Thymeleaf 3.1.2:** Server-side template engine
- **Bootstrap 5.3.3:** CSS framework
- **Bootstrap Icons 1.11.3:** Icon library
- **jQuery 3.7.1:** JavaScript library
- **Chart.js 4.4.0:** Data visualization

Database:

- **MySQL 9.2:** Primary database
- **Flyway 10.4.1:** Database migration tool
- **HikariCP:** Connection pooling

External APIs:

- **Google Sheets API v4:** Attendance tracking
- **Google OAuth2:** Social login

Build & Development:

- **Gradle 9.2.1:** Build automation
- **Lombok:** Boilerplate code reduction
- **Spring Boot DevTools:** Development utilities

5.5 Communication Flow

User Request Flow:

```
1. User → Browser (HTTP Request)
2. Browser → Spring DispatcherServlet
3. DispatcherServlet → Controller
4. Controller → Service Layer
5. Service → Repository
6. Repository → Database
7. Database → Repository (Data)
8. Repository → Service (Entity)
9. Service → Controller (DTO)
10. Controller → Thymeleaf (Model)
11. Thymeleaf → Browser (HTML)
12. Browser → User (Rendered Page)
```

Authentication Flow:

```
1. User submits credentials
2. Spring Security intercepts request
3. CustomUserDetailsService loads user
4. Password verification (BCrypt)
5. Authentication success/failure handler
```

```
6. Session creation
7. Redirect to role-specific dashboard
```

---

6. DATABASE DESIGN

6.1 Entity-Relationship Diagram

Core Entities and Relationships:

```
┌──────────────┐
│   User       │
│ (Abstract)   │
└──────────────┘
      △
      │ (Inheritance)
      │
┌─────┼─────┬─────┐
│     │     │     │
Student Lecturer Admin
│     │     │     │
└─────┴─────┴─────┘

User ↔ Role (Many-to-Many)
Student ↔ Enrollment ↔ Course (Many-to-Many through Enrollment)
Lecturer ↔ Course (One-to-Many)
Course ↔ Schedule (One-to-Many)
Schedule ↔ Classroom (Many-to-One)
Schedule ↔ TimeSlot (Many-to-One)
Course ↔ Department (Many-to-One)
Course ↔ Semester (Many-to-One)
Student ↔ Attendance ↔ Course (Through Attendance)
Course ↔ CourseSheet (One-to-One)
```

6.2 Database Tables

User Management Tables:

1. users

- id (BIGINT, PK, AUTO_INCREMENT)
- username (VARCHAR(50), UNIQUE)
- email (VARCHAR(100), UNIQUE)
- password (VARCHAR(255))
- first_name (VARCHAR(100))
- last_name (VARCHAR(100))
- phone_number (VARCHAR(20))
- profile_picture (VARCHAR(255))
- is_active (BOOLEAN)

- created_at (TIMESTAMP)
- updated_at (TIMESTAMP)

2. roles

- id (BIGINT, PK, AUTO_INCREMENT)
- name (VARCHAR(50), UNIQUE)
- description (VARCHAR(255))

3. user_roles (Join Table)

- user_id (BIGINT, FK → users.id)
- role_id (BIGINT, FK → roles.id)
- PRIMARY KEY (user_id, role_id)

4. students (Extends users)

- id (BIGINT, PK, FK → users.id)
- student_id (VARCHAR(20), UNIQUE)
- major (VARCHAR(100))
- year_level (INT)
- gpa (DECIMAL(3,2))
- enrollment_date (DATE)

5. lecturers (Extends users)

- id (BIGINT, PK, FK → users.id)
- employee_id (VARCHAR(20), UNIQUE)
- specialization (VARCHAR(100))
- office_room (VARCHAR(50))
- department_id (BIGINT, FK → departments.id)

6. admins (Extends users)

- id (BIGINT, PK, FK → users.id)
- admin_id (VARCHAR(20), UNIQUE)
- department (VARCHAR(100))
- admin_level (VARCHAR(50))

Academic Tables:

7. departments

- id (BIGINT, PK, AUTO_INCREMENT)
- department_name (VARCHAR(100), UNIQUE)
- department_code (VARCHAR(20), UNIQUE)
- description (TEXT)

8. semesters

- id (BIGINT, PK, AUTO_INCREMENT)
- semester_name (VARCHAR(50))

- start_date (DATE)
- end_date (DATE)
- is_active (BOOLEAN)

9. courses

- id (BIGINT, PK, AUTO_INCREMENT)
- course_code (VARCHAR(20), UNIQUE)
- course_name (VARCHAR(100))
- description (TEXT)
- credits (INT)
- max_students (INT)
- status (ENUM: ACTIVE, INACTIVE, ARCHIVED)
- lecturer_id (BIGINT, FK → lecturers.id)
- department_id (BIGINT, FK → departments.id)
- semester_id (BIGINT, FK → semesters.id)
- created_at (TIMESTAMP)
- updated_at (TIMESTAMP)

10. enrollments

- id (BIGINT, PK, AUTO_INCREMENT)
- student_id (BIGINT, FK → students.id)
- course_id (BIGINT, FK → courses.id)
- status (ENUM: PENDING, APPROVED, REJECTED, DROPPED)
- enrolled_at (TIMESTAMP)
- approved_at (TIMESTAMP)
- grade (VARCHAR(5))

Scheduling Tables:

11. classrooms

- id (BIGINT, PK, AUTO_INCREMENT)
- room_number (VARCHAR(50))
- building (VARCHAR(100))
- capacity (INT)
- has_projector (BOOLEAN)
- has_whiteboard (BOOLEAN)
- is_available (BOOLEAN)

12. time_slots

- id (BIGINT, PK, AUTO_INCREMENT)
- start_time (TIME)
- end_time (TIME)
- day_of_week (ENUM: MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY)

13. schedules

- id (BIGINT, PK, AUTO_INCREMENT)
- course_id (BIGINT, FK → courses.id)
- classroom_id (BIGINT, FK → classrooms.id)
- time_slot_id (BIGINT, FK → time_slots.id)
- effective_date (DATE)
- expiry_date (DATE)

Attendance Tables:

14. attendance

- id (BIGINT, PK, AUTO_INCREMENT)
- student_id (BIGINT, FK → students.id)
- course_id (BIGINT, FK → courses.id)
- lecturer_id (BIGINT, FK → lecturers.id)
- attendance_date (DATE)
- status (ENUM: PRESENT, ABSENT, LATE, EXCUSED)
- notes (TEXT)
- created_at (TIMESTAMP)
- UNIQUE KEY (student_id, course_id, attendance_date)

15. course_sheets

- id (BIGINT, PK, AUTO_INCREMENT)
- course_id (BIGINT, FK → courses.id, UNIQUE)
- spreadsheet_id (VARCHAR(255))
- sheet_name (VARCHAR(100))
- created_at (TIMESTAMP)
- updated_at (TIMESTAMP)

Audit Tables:

16. user_sessions

- id (BIGINT, PK, AUTO_INCREMENT)
- user_id (BIGINT, FK → users.id)
- session_token (VARCHAR(255), UNIQUE)
- ip_address (VARCHAR(50))
- user_agent (VARCHAR(255))
- created_at (TIMESTAMP)
- expires_at (TIMESTAMP)

17. login_history

- id (BIGINT, PK, AUTO_INCREMENT)
- user_id (BIGINT, FK → users.id)
- login_time (TIMESTAMP)
- logout_time (TIMESTAMP)
- ip_address (VARCHAR(50))
- success (BOOLEAN)

18. user_activity_audit

- id (BIGINT, PK, AUTO_INCREMENT)
- user_id (BIGINT, FK → users.id)
- action (VARCHAR(100))
- entity_type (VARCHAR(50))
- entity_id (BIGINT)
- old_value (TEXT)
- new_value (TEXT)
- timestamp (TIMESTAMP)

6.3 Database Migration Strategy

Flyway Migration Files:

V1__Create_base_tables.sql Purpose: Create all core tables

- Users and role tables
- Student, Lecturer, Admin tables
- Course and enrollment tables
- Schedule and classroom tables

V2__Insert_initial_data.sql Purpose: Insert default data

- Default roles (ROLE_ADMIN, ROLE_LECTURER, ROLE_STUDENT)
- Sample departments
- Default admin account

V3__Insert_courses_and_schedules.sql Purpose: Sample data

- Example courses
- Sample schedules
- Time slots

V4__Create_session_and_audit_tables.sql Purpose: Security and auditing

- user_sessions table
- login_history table
- user_activity_audit table

V5__Add_profile_picture_to_users.sql Purpose: Profile feature

- Add profile_picture column to users table

V6__Add_admin_level_and_department_to_admins.sql Purpose: Admin hierarchy

- Add admin_level column
- Add department column to admins

V7__Create_attendance_table.sql Purpose: Attendance tracking

- Create attendance table

- Add indexes for performance

V8__Create_course_sheets_table.sql Purpose: Google Sheets integration

- Create course_sheets table
- Link courses to Google Sheets

6.4 Database Indexing Strategy

Primary Indexes:

- All primary keys have clustered indexes by default

Foreign Key Indexes:

- Indexes on all foreign key columns for join performance

Unique Indexes:

- users.username
- users.email
- students.student_id
- lecturers.employee_id
- admins.admin_id
- courses.course_code
- departments.department_code

Composite Indexes:

- (student_id, course_id, attendance_date) on attendance table
- (user_id, role_id) on user_roles table
- (course_id, time_slot_id) on schedules table

6.5 Database Constraints

Foreign Key Constraints:

- CASCADE on delete for dependent records
- RESTRICT on delete for referenced records

Check Constraints:

- year_level between 1 and 4
- gpa between 0.00 and 4.00
- credits > 0
- max_students > 0
- capacity > 0

Not Null Constraints:

- All required fields marked NOT NULL
- Email, username always required

- Timestamps auto-generated

---

7. APPLICATION DESIGN PATTERNS

7.1 Architectural Patterns

1. Layered Architecture

```
Presentation Layer → Controller Layer → Service Layer → Repository Layer → Data Layer
```

Benefits:

- Clear separation of concerns
- Easy to maintain and test
- Scalable structure
- Reusable components

2. MVC Pattern

Model-View-Controller separation:

- **Model:** JPA entities (User, Course, Enrollment, etc.)
- **View:** Thymeleaf templates (HTML with dynamic content)
- **Controller:** Spring MVC controllers (handle HTTP requests)

3. Repository Pattern

Spring Data JPA provides:

- CRUD operations without boilerplate code
- Custom query methods using method names
- @Query annotations for complex queries
- Pagination and sorting support

Example:

```
public interface StudentRepository extends JpaRepository<Student, Long> {
    Optional<Student> findByStudentId(String studentId);
    List<Student> findByMajor(String major);
    List<Student> findByYearLevel(Integer yearLevel);
}
```

4. Service Layer Pattern

Business logic encapsulation:

- Transaction management with @Transactional

- Validation before persistence
- DTO conversion
- Error handling

Example structure:

```
@Service
@Transactional
public class StudentServiceImpl implements StudentService {
    private final StudentRepository studentRepository;

    // Business methods
}
```

5. DTO Pattern

Data Transfer Objects:

- Separate API contracts from domain models
- Reduce over-fetching
- Validation annotations
- Security (hide sensitive data)

7.2 Design Patterns Used

1. Singleton Pattern

- Spring beans are singletons by default
- Service classes, repositories, configurations

2. Factory Pattern

- Used in authentication process
- User creation based on role type

3. Strategy Pattern

- Different authentication strategies (Form, OAuth2)
- Role-based access control strategies

4. Observer Pattern

- Event listeners for audit logging
- Session management events

5. Template Method Pattern

- Base service classes with common operations
- Specialized implementations in child classes

6. Dependency Injection Pattern

- Constructor injection (preferred)
- Field injection (for configuration)
- Interface-based dependencies

7. Builder Pattern

- Entity builders (Lombok @Builder)
- Query builders in repositories

7.3 SOLID Principles Implementation

S - Single Responsibility Principle:

- Each class has one reason to change
- Separate services for different domains
- Controllers only handle HTTP requests

O - Open/Closed Principle:

- Interfaces for services
- Extension through inheritance
- Configuration-based behavior changes

L - Liskov Substitution Principle:

- User hierarchy (Student, Lecturer, Admin extend User)
- Interface implementations are interchangeable

I - Interface Segregation Principle:

- Specific service interfaces
- Clients depend only on methods they use
- Role-specific controllers

D - Dependency Inversion Principle:

- Depend on abstractions (interfaces)
- High-level modules independent of low-level
- Dependency injection throughout

7.4 Code Organization

Package Structure:

```
com.university.courseenrollment.demogradle/
├── config/                    # Configuration classes
├── controller/
│   ├── api/                  # REST API controllers
│   └── web/                  # Web page controllers
├── dto/                      # Data transfer objects
├── enums/                    # Enumeration types
├── exception/                # Custom exceptions
```

```
├── model/entity/             # JPA entities
├── repository/               # Data access layer
├── security/                 # Security components
├── service/                  # Business logic
│   ├── auth/                # Authentication services
│   ├── course/              # Course services
│   ├── schedule/            # Schedule services
│   ├── attendance/          # Attendance services
│   ├── audit/               # Audit services
│   ├── session/             # Session services
│   └── googlesheets/        # Google Sheets services
├── util/                     # Utility classes
└── validator/                # Custom validators
```

---

8. SECURITY ARCHITECTURE

8.1 Authentication System

Authentication Methods:

1. Form-Based Authentication

- Username/password login
- BCrypt password encryption
- Custom UserDetailsService
- Session-based authentication

2. OAuth2 Authentication

- Google OAuth2 provider
- Social login support
- Automatic user registration
- Profile data sync

Authentication Flow:

```
1. User submits credentials
2. Spring Security filter intercepts
3. AuthenticationManager processes
4. UserDetailsService loads user from database
5. PasswordEncoder verifies password
6. Authentication object created
7. SecurityContext updated
8. Session created
9. Success/Failure handler redirects
```

8.2 Authorization System

Role-Based Access Control (RBAC):

Roles:

- ROLE_ADMIN: Full system access
- ROLE_LECTURER: Course and attendance management
- ROLE_STUDENT: Enrollment and viewing

Method-Level Security:

```
@PreAuthorize("hasRole('ADMIN')")
public String adminDashboard() { ... }

@PreAuthorize("hasAnyRole('ADMIN', 'LECTURER')")
public String viewSchedule() { ... }

@PreAuthorize("hasRole('STUDENT')")
public String enrollCourse() { ... }
```

URL-Based Security:

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/admin/**").hasRole("ADMIN")
    .requestMatchers("/lecturer/**").hasAnyRole("LECTURER", "ADMIN")
    .requestMatchers("/student/**").hasRole("STUDENT")
    .anyRequest().authenticated()
)
```

8.3 Security Features

1. Password Security

- BCrypt encryption (strength 10)
- Salt generation per password
- No plaintext storage
- Password validation rules

2. Session Management

- Server-side session storage
- Session timeout (30 minutes default)
- Session fixation protection
- Concurrent session control

3. CSRF Protection

- Currently disabled for API development
- Should be enabled in production
- Token-based validation

4. XSS Protection

- Thymeleaf automatic escaping
- Input validation
- Content Security Policy headers

5. SQL Injection Prevention

- Prepared statements (JPA)
- Parameter binding
- Input validation

6. Authentication Events

- Login success/failure tracking
- Account lockout after failed attempts
- Activity audit logging

8.4 Security Configuration

SecurityConfig.java:

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true)
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/", "/auth/**", "/css/**", "/js/**").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .requestMatchers("/lecturer/**").hasAnyRole("LECTURER", "ADMIN")
                .requestMatchers("/student/**").hasRole("STUDENT")
                .anyRequest().authenticated()
            )
            .formLogin(form -> form
                .loginPage("/auth/login")
                .successHandler(customSuccessHandler)
                .failureHandler(customFailureHandler)
            )
            .oauth2Login(oauth2 -> oauth2
                .loginPage("/auth/login")
                .successHandler(oauth2SuccessHandler)
            )
            .logout(logout -> logout
                .logoutUrl("/auth/logout")
                .logoutSuccessUrl("/auth/login?logout=true")
            );

        return http.build();
```

```
    }
}
```

8.5 Data Protection

Sensitive Data Handling:

- Passwords never logged
- Personal information encrypted
- Profile pictures stored separately
- Database credentials externalized

Audit Trail:

- All CRUD operations logged
- User activity tracking
- Login/logout history
- Changes tracked (old vs new values)

Access Control:

- Users can only access their own data
- Admins can access all data
- Lecturers can access assigned courses
- Students can access enrolled courses

PART III: IMPLEMENTATION

---

9. TECHNOLOGY STACK DETAILS

9.1 Backend Technologies

Spring Framework Stack:

- **Spring Boot 3.4.1:** Core application framework
- **Spring MVC:** Web application framework
- **Spring Security 6.2.1:** Authentication and authorization
- **Spring Data JPA:** Data persistence layer
- **Spring AOP:** Aspect-oriented programming for cross-cutting concerns

Data Access:

- **Hibernate 6.6.4:** JPA implementation and ORM
- **Flyway 10.4.1:** Database version control and migrations
- **HikariCP:** High-performance JDBC connection pooling

Utilities:

- **Lombok:** Reduce boilerplate code with annotations
- **Jackson:** JSON serialization/deserialization
- **SLF4J + Logback:** Logging framework

9.2 Frontend Technologies

Template Engine:

- **Thymeleaf 3.1.2:** Server-side Java template engine
  - Natural templating
  - Spring integration
  - Fragment support
  - Layout dialect

CSS Framework:

- **Bootstrap 5.3.3:** Responsive design framework
  - Grid system
  - Components
  - Utilities
  - Responsive breakpoints

JavaScript Libraries:

- **jQuery 3.7.1:** DOM manipulation and AJAX
- **Chart.js 4.4.0:** Interactive charts and graphs
- **Bootstrap Icons 1.11.3:** Icon library

9.3 Database Technologies

Database Management System:

- **MySQL 9.2:** Relational database
  - ACID compliance
  - Transaction support
  - Full-text search
  - JSON support

Connection Pooling:

- **HikariCP:** Fast, reliable connection pooling
  - Maximum pool size: 10
  - Minimum idle: 5
  - Connection timeout: 30 seconds

9.4 External APIs

Google Cloud APIs:

- **Google Sheets API v4:** Attendance tracking integration
- **Google OAuth2:** Social authentication

- **Google Drive API:** Sheet management

Authentication:

- Service account authentication
- OAuth2 client credentials
- API key management

9.5 Build and Development Tools

Build System:

- **Gradle 9.2.1:** Build automation tool
  - Dependency management
  - Multi-project builds
  - Custom tasks

Development Tools:

- **Spring Boot DevTools:** Hot reload and fast restart
- **Git:** Version control system
- **IntelliJ IDEA:** Primary IDE (recommended)

9.6 Version Matrix

Component	Version	Release Date
Java	21.0.5	October 2024
Spring Boot	3.4.1	December 2024
Spring Security	6.2.1	January 2024
Hibernate	6.6.4	November 2024
MySQL	9.2	October 2024
Thymeleaf	3.1.2	May 2023
Bootstrap	5.3.3	February 2024
Gradle	9.2.1	May 2024

---

10. BACKEND IMPLEMENTATION

10.1 Entity Layer

Base Entity Structure:

All entities extend a common base with audit fields:

```
@MappedSuperclass
public abstract class BaseEntity {
```

```
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @CreatedDate
    private LocalDateTime createdAt;

    @LastModifiedDate
    private LocalDateTime updatedAt;
}
```

User Hierarchy:

```
@Entity
@Table(name = "users")
@Inheritance(strategy = InheritanceType.JOINED)
public class User extends BaseEntity {
    private String username;
    private String email;
    private String password;
    private String firstName;
    private String lastName;
    private String phoneNumber;
    private String profilePicture;
    private Boolean isActive;

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(name = "user_roles",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "role_id"))
    private Set<Role> roles;
}
```

Student Entity:

```
@Entity
@Table(name = "students")
@PrimaryKeyJoinColumn(name = "id")
public class Student extends User {
    private String studentId;
    private String major;
    private Integer yearLevel;
    private BigDecimal gpa;
    private LocalDate enrollmentDate;

    @OneToMany(mappedBy = "student")
    private List<Enrollment> enrollments;

    @OneToMany(mappedBy = "student")
```

```
    private List<Attendance> attendances;
}
```

Lecturer Entity:

```
@Entity
@Table(name = "lecturers")
@PrimaryKeyJoinColumn(name = "id")
public class Lecturer extends User {
    private String employeeId;
    private String specialization;
    private String officeRoom;

    @ManyToOne
    @JoinColumn(name = "department_id")
    private Department department;

    @OneToMany(mappedBy = "lecturer")
    private List<Course> courses;
}
```

Course Entity:

```
@Entity
@Table(name = "courses")
public class Course extends BaseEntity {
    private String courseCode;
    private String courseName;
    private String description;
    private Integer credits;
    private Integer maxStudents;

    @Enumerated(EnumType.STRING)
    private CourseStatus status;

    @ManyToOne
    @JoinColumn(name = "lecturer_id")
    private Lecturer lecturer;

    @ManyToOne
    @JoinColumn(name = "department_id")
    private Department department;

    @ManyToOne
    @JoinColumn(name = "semester_id")
    private Semester semester;

    @OneToMany(mappedBy = "course")
    private List<Enrollment> enrollments;
```

```
    @OneToMany(mappedBy = "course")
    private List<Schedule> schedules;

    @OneToOne(mappedBy = "course")
    private CourseSheet courseSheet;
}
```

Enrollment Entity:

```
@Entity
@Table(name = "enrollments")
public class Enrollment extends BaseEntity {
    @ManyToOne
    @JoinColumn(name = "student_id")
    private Student student;

    @ManyToOne
    @JoinColumn(name = "course_id")
    private Course course;

    @Enumerated(EnumType.STRING)
    private EnrollmentStatus status;

    private LocalDateTime enrolledAt;
    private LocalDateTime approvedAt;
    private String grade;
}
```

10.2 Repository Layer

Standard Repository Pattern:

```
@Repository
public interface StudentRepository extends JpaRepository<Student, Long> {
    Optional<Student> findByStudentId(String studentId);
    Optional<Student> findByUsername(String username);
    Optional<Student> findByEmail(String email);
    List<Student> findByMajor(String major);
    List<Student> findByYearLevel(Integer yearLevel);

    @Query("SELECT s FROM Student s WHERE s.isActive = true")
    List<Student> findAllActiveStudents();
}
```

Complex Queries:

```
@Repository
public interface EnrollmentRepository extends JpaRepository<Enrollment, Long> {
    List<Enrollment> findByStudentId(Long studentId);
    List<Enrollment> findByCourseId(Long courseId);
    List<Enrollment> findByStatus(EnrollmentStatus status);

    @Query("SELECT e FROM Enrollment e WHERE e.student.id = :studentId AND
e.status = :status")
    List<Enrollment> findByStudentIdAndStatus(@Param("studentId") Long studentId,
                                              @Param("status") EnrollmentStatus
status);

    @Query("SELECT COUNT(e) FROM Enrollment e WHERE e.course.id = :courseId AND
e.status = 'APPROVED'")
    Long countApprovedEnrollmentsByCourse(@Param("courseId") Long courseId);

    boolean existsByStudentIdAndCourseId(Long studentId, Long courseId);
}
```

10.3 Service Layer

Service Interface Pattern:

```
public interface StudentService {
    StudentDTO createStudent(StudentDTO studentDTO);
    StudentDTO updateStudent(Long id, StudentDTO studentDTO);
    StudentDTO getStudentById(Long id);
    StudentDTO getStudentByStudentId(String studentId);
    List<StudentDTO> getAllStudents();
    void deleteStudent(Long id);
    List<CourseDTO> getEnrolledCourses(Long studentId);
}
```

Service Implementation:

```
@Service
@Transactional
public class StudentServiceImpl implements StudentService {

    private final StudentRepository studentRepository;
    private final PasswordEncoder passwordEncoder;

    public StudentServiceImpl(StudentRepository studentRepository,
                              PasswordEncoder passwordEncoder) {
        this.studentRepository = studentRepository;
        this.passwordEncoder = passwordEncoder;
    }
```

```
    @Override
    public StudentDTO createStudent(StudentDTO studentDTO) {
        // Validation
        validateStudentDTO(studentDTO);

        // Check for duplicates
        if
(studentRepository.findByStudentId(studentDTO.getStudentId()).isPresent()) {
            throw new DuplicateResourceException("Student ID already exists");
        }

        // Create entity
        Student student = new Student();
        student.setStudentId(studentDTO.getStudentId());
        student.setUsername(studentDTO.getUsername());
        student.setEmail(studentDTO.getEmail());
        student.setPassword(passwordEncoder.encode(studentDTO.getPassword()));
        student.setFirstName(studentDTO.getFirstName());
        student.setLastName(studentDTO.getLastName());
        student.setMajor(studentDTO.getMajor());
        student.setYearLevel(studentDTO.getYearLevel());
        student.setIsActive(true);

        // Save
        Student savedStudent = studentRepository.save(student);

        // Convert to DTO and return
        return convertToDTO(savedStudent);
    }

    @Override
    @Transactional(readOnly = true)
    public StudentDTO getStudentById(Long id) {
        Student student = studentRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Student not
found"));
        return convertToDTO(student);
    }

    private StudentDTO convertToDTO(Student student) {
        StudentDTO dto = new StudentDTO();
        dto.setId(student.getId());
        dto.setStudentId(student.getStudentId());
        dto.setUsername(student.getUsername());
        dto.setEmail(student.getEmail());
        dto.setFirstName(student.getFirstName());
        dto.setLastName(student.getLastName());
        dto.setMajor(student.getMajor());
        dto.setYearLevel(student.getYearLevel());
        return dto;
    }
}
```

10.4 Controller Layer

Web Controller Example:

```
@Controller
@RequestMapping("/student")
public class StudentDashboardController {

    private final StudentService studentService;
    private final EnrollmentService enrollmentService;

    @GetMapping("/dashboard")
    public String dashboard(Model model, Authentication authentication) {
        String username = authentication.getName();
        StudentDTO student = studentService.getStudentByUsername(username);

        // Get enrolled courses
        List<CourseDTO> enrolledCourses = enrollmentService
            .getEnrolledCoursesByStudentId(student.getId());

        // Get statistics
        model.addAttribute("student", student);
        model.addAttribute("enrolledCourses", enrolledCourses);
        model.addAttribute("totalCredits",
calculateTotalCredits(enrolledCourses));

        return "dashboard/student-dashboard";
    }

    @GetMapping("/profile")
    public String viewProfile(Model model, Authentication authentication) {
        String username = authentication.getName();
        StudentDTO student = studentService.getStudentByUsername(username);
        model.addAttribute("student", student);
        return "student/profile";
    }
}
```

REST Controller Example:

```
@RestController
@RequestMapping("/api/courses")
public class CourseRestController {

    private final CourseService courseService;

    @GetMapping
    public ResponseEntity<List<CourseDTO>> getAllCourses() {
        List<CourseDTO> courses = courseService.getAllCourses();
        return ResponseEntity.ok(courses);
```

```
    }

    @GetMapping("/{id}")
    public ResponseEntity<CourseDTO> getCourseById(@PathVariable Long id) {
        CourseDTO course = courseService.getCourseById(id);
        return ResponseEntity.ok(course);
    }

    @PostMapping
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<CourseDTO> createCourse(@Valid @RequestBody CourseDTO
courseDTO) {
        CourseDTO created = courseService.createCourse(courseDTO);
        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    }

    @PutMapping("/{id}")
    @PreAuthorize("hasAnyRole('ADMIN', 'LECTURER')")
    public ResponseEntity<CourseDTO> updateCourse(@PathVariable Long id,
                                                  @Valid @RequestBody CourseDTO
courseDTO) {
        CourseDTO updated = courseService.updateCourse(id, courseDTO);
        return ResponseEntity.ok(updated);
    }

    @DeleteMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<Void> deleteCourse(@PathVariable Long id) {
        courseService.deleteCourse(id);
        return ResponseEntity.noContent().build();
    }
}
```

10.5 Exception Handling

Global Exception Handler:

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse>
handleResourceNotFound(ResourceNotFoundException ex) {
        ErrorResponse error = new ErrorResponse(
            HttpStatus.NOT_FOUND.value(),
            ex.getMessage(),
            LocalDateTime.now()
        );
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }
```

```
    @ExceptionHandler(DuplicateResourceException.class)
    public ResponseEntity<ErrorResponse>
handleDuplicateResource(DuplicateResourceException ex) {
        ErrorResponse error = new ErrorResponse(
            HttpStatus.CONFLICT.value(),
            ex.getMessage(),
            LocalDateTime.now()
        );
        return ResponseEntity.status(HttpStatus.CONFLICT).body(error);
    }

    @ExceptionHandler(ScheduleConflictException.class)
    public ResponseEntity<ErrorResponse>
handleScheduleConflict(ScheduleConflictException ex) {
        ErrorResponse error = new ErrorResponse(
            HttpStatus.BAD_REQUEST.value(),
            ex.getMessage(),
            LocalDateTime.now()
        );
        return ResponseEntity.badRequest().body(error);
    }

    @ExceptionHandler(UnauthorizedAccessException.class)
    public ResponseEntity<ErrorResponse>
handleUnauthorizedAccess(UnauthorizedAccessException ex) {
        ErrorResponse error = new ErrorResponse(
            HttpStatus.FORBIDDEN.value(),
            ex.getMessage(),
            LocalDateTime.now()
        );
        return ResponseEntity.status(HttpStatus.FORBIDDEN).body(error);
    }
}
```

10.6 Validation

DTO Validation:

```
@Data
public class StudentDTO {
    private Long id;

    @NotBlank(message = "Student ID is required")
    @Size(min = 5, max = 20, message = "Student ID must be between 5 and 20
characters")
    private String studentId;

    @NotBlank(message = "Username is required")
    @Size(min = 3, max = 50, message = "Username must be between 3 and 50
characters")
    private String username;
```

```
    @NotBlank(message = "Email is required")
    @Email(message = "Invalid email format")
    private String email;

    @NotBlank(message = "First name is required")
    private String firstName;

    @NotBlank(message = "Last name is required")
    private String lastName;

    @Min(value = 1, message = "Year level must be at least 1")
    @Max(value = 4, message = "Year level cannot exceed 4")
    private Integer yearLevel;

    @DecimalMin(value = "0.00", message = "GPA cannot be negative")
    @DecimalMax(value = "4.00", message = "GPA cannot exceed 4.00")
    private BigDecimal gpa;
}
```

Custom Validators:

```
@Component
public class CourseValidator {

    private final CourseRepository courseRepository;

    public void validateCourseCreation(CourseDTO courseDTO) {
        // Check course code uniqueness
        if (courseRepository.existsByCourseCode(courseDTO.getCourseCode())) {
            throw new DuplicateResourceException("Course code already exists");
        }

        // Validate credits
        if (courseDTO.getCredits() <= 0 || courseDTO.getCredits() > 6) {
            throw new IllegalArgumentException("Credits must be between 1 and 6");
        }

        // Validate max students
        if (courseDTO.getMaxStudents() <= 0) {
            throw new IllegalArgumentException("Max students must be positive");
        }
    }

    public void validateEnrollment(Long studentId, Long courseId) {
        Course course = courseRepository.findById(courseId)
            .orElseThrow(() -> new ResourceNotFoundException("Course not found"));

        // Check if course is active
        if (course.getStatus() != CourseStatus.ACTIVE) {
            throw new IllegalStateException("Course is not active for
enrollment");
```

```
        }

        // Check capacity
        long enrolledCount =
enrollmentRepository.countApprovedEnrollmentsByCourse(courseId);
        if (enrolledCount >= course.getMaxStudents()) {
            throw new IllegalStateException("Course is full");
        }

        // Check duplicate enrollment
        if (enrollmentRepository.existsByStudentIdAndCourseId(studentId,
courseId)) {
            throw new DuplicateResourceException("Already enrolled in this
course");
        }
    }
}
```

10.7 Aspect-Oriented Programming

Audit Logging Aspect:

```
@Aspect
@Component
public class AuditAspect {

    private final AuditService auditService;

    @Around("@annotation(Audited)")
    public Object auditMethod(ProceedingJoinPoint joinPoint) throws Throwable {
        // Get current user
        Authentication authentication =
SecurityContextHolder.getContext().getAuthentication();
        String username = authentication != null ? authentication.getName() :
"anonymous";

        // Get method information
        MethodSignature signature = (MethodSignature) joinPoint.getSignature();
        String methodName = signature.getName();
        String className = signature.getDeclaringType().getSimpleName();

        // Execute method
        Object result;
        try {
            result = joinPoint.proceed();

            // Log success
            auditService.logAction(
                username,
                className + "." + methodName,
                "SUCCESS",
```

```
                Arrays.toString(joinPoint.getArgs())
            );
        } catch (Exception e) {
            // Log failure
            auditService.logAction(
                username,
                className + "." + methodName,
                "FAILURE",
                e.getMessage()
            );
            throw e;
        }

        return result;
    }
}
```

10.8 Configuration Classes

Security Configuration:

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true)
public class SecurityConfig {

    private final CustomUserDetailsService userDetailsService;
    private final CustomAuthenticationSuccessHandler successHandler;
    private final CustomAuthenticationFailureHandler failureHandler;
    private final OAuth2LoginSuccessHandler oauth2SuccessHandler;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/", "/auth/**", "/css/**", "/js/**",
"/images/**").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .requestMatchers("/lecturer/**").hasAnyRole("LECTURER", "ADMIN")
                .requestMatchers("/student/**").hasRole("STUDENT")
                .anyRequest().authenticated()
            )
            .formLogin(form -> form
                .loginPage("/auth/login")
                .loginProcessingUrl("/auth/login")
                .successHandler(successHandler)
                .failureHandler(failureHandler)
                .permitAll()
            )
            .oauth2Login(oauth2 -> oauth2
```

```
                .loginPage("/auth/login")
                .successHandler(oauth2SuccessHandler)
            )
            .logout(logout -> logout
                .logoutUrl("/auth/logout")
                .logoutSuccessUrl("/auth/login?logout=true")
                .invalidateHttpSession(true)
                .deleteCookies("JSESSIONID")
                .permitAll()
            )
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED)
                .maximumSessions(1)
                .expiredUrl("/auth/login?expired=true")
            )
            .csrf(csrf -> csrf.disable()); // Enable in production

        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(10);
    }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration
config)
            throws Exception {
        return config.getAuthenticationManager();
    }
}
```

Database Configuration:

```
@Configuration
@EnableJpaRepositories(basePackages =
"com.university.courseenrollment.demogradle.repository")
@EnableJpaAuditing
public class DatabaseConfig {

    @Bean
    public DataSource dataSource() {
        HikariConfig config = new HikariConfig();
        config.setJdbcUrl("jdbc:mysql://localhost:3306/course_enrollment_db");
        config.setUsername("root");
        config.setPassword("password");
        config.setMaximumPoolSize(10);
        config.setMinimumIdle(5);
        config.setConnectionTimeout(30000);
        config.setIdleTimeout(600000);
        config.setMaxLifetime(1800000);
```

```
        return new HikariDataSource(config);
    }

    @Bean
    public LocalContainerEntityManagerFactoryBean entityManagerFactory(DataSource
dataSource) {
        LocalContainerEntityManagerFactoryBean em = new
LocalContainerEntityManagerFactoryBean();
        em.setDataSource(dataSource);

em.setPackagesToScan("com.university.courseenrollment.demogradle.model.entity");

        HibernateJpaVendorAdapter vendorAdapter = new HibernateJpaVendorAdapter();
        vendorAdapter.setShowSql(true);
        vendorAdapter.setGenerateDdl(false);
        em.setJpaVendorAdapter(vendorAdapter);

        Properties properties = new Properties();
        properties.setProperty("hibernate.dialect",
"org.hibernate.dialect.MySQLDialect");
        properties.setProperty("hibernate.hbm2ddl.auto", "validate");
        properties.setProperty("hibernate.format_sql", "true");
        em.setJpaProperties(properties);

        return em;
    }
}
```

---

11. FRONTEND IMPLEMENTATION

11.1 Template Structure

Main Layout (layout/main.html):

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title th:text="${pageTitle} ?: 'University Enrollment
System'">University</title>

    <!-- Bootstrap CSS -->
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
          rel="stylesheet">
    <link href="https://cdn.jsdelivr.net/npm/bootstrap-
icons@1.11.3/font/bootstrap-icons.css"
```

```
          rel="stylesheet">

    <!-- Custom CSS -->
    <link th:href="@{/css/style.css}" rel="stylesheet">
</head>
<body>
    <!-- Header -->
    <div th:replace="~{layout/header :: header}"></div>

    <div class="container-fluid">
        <div class="row">
            <!-- Sidebar -->
            <div th:replace="~{layout/sidebar :: sidebar}"></div>

            <!-- Main Content -->
            <main class="col-md-9 ms-sm-auto col-lg-10 px-md-4">
                <!-- Alert Messages -->
                <div th:if="${successMessage}" class="alert alert-success alert-
dismissible fade show">
                    <span th:text="${successMessage}"></span>
                    <button type="button" class="btn-close" data-bs-
dismiss="alert"></button>
                </div>

                <div th:if="${errorMessage}" class="alert alert-danger alert-
dismissible fade show">
                    <span th:text="${errorMessage}"></span>
                    <button type="button" class="btn-close" data-bs-
dismiss="alert"></button>
                </div>

                <!-- Page Content -->
                <div layout:fragment="content">
                    <!-- Content will be inserted here -->
                </div>
            </main>
        </div>
    </div>

    <!-- Footer -->
    <div th:replace="~{layout/footer :: footer}"></div>

    <!-- Bootstrap JS -->
    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
></script>
    <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
    <script
src="https://cdn.jsdelivr.net/npm/chart.js@4.4.0/dist/chart.umd.min.js"></script>
</body>
</html>
```

11.2 Dashboard Implementation

Student Dashboard:

```
<div layout:decorate="~{layout/main}">
    <div layout:fragment="content">
        <h1 class="h2">Student Dashboard</h1>

        <!-- Statistics Cards -->
        <div class="row mt-4">
            <div class="col-md-3">
                <div class="card text-white bg-primary">
                    <div class="card-body">
                        <h5 class="card-title">Enrolled Courses</h5>
                        <h2 th:text="${enrolledCoursesCount}">0</h2>
                    </div>
                </div>
            </div>

            <div class="col-md-3">
                <div class="card text-white bg-success">
                    <div class="card-body">
                        <h5 class="card-title">Total Credits</h5>
                        <h2 th:text="${totalCredits}">0</h2>
                    </div>
                </div>
            </div>

            <div class="col-md-3">
                <div class="card text-white bg-warning">
                    <div class="card-body">
                        <h5 class="card-title">Current GPA</h5>
                        <h2 th:text="${gpa}">0.00</h2>
                    </div>
                </div>
            </div>

            <div class="col-md-3">
                <div class="card text-white bg-info">
                    <div class="card-body">
                        <h5 class="card-title">Attendance Rate</h5>
                        <h2 th:text="${attendanceRate} + '%'">0%</h2>
                    </div>
                </div>
            </div>
        </div>

        <!-- Enrolled Courses Table -->
        <div class="card mt-4">
            <div class="card-header">
                <h5>My Courses</h5>
            </div>
```

```
            <div class="card-body">
                <table class="table table-striped">
                    <thead>
                        <tr>
                            <th>Course Code</th>
                            <th>Course Name</th>
                            <th>Lecturer</th>
                            <th>Credits</th>
                            <th>Status</th>
                        </tr>
                    </thead>
                    <tbody>
                        <tr th:each="course : ${enrolledCourses}">
                            <td th:text="${course.courseCode}">CS101</td>
                            <td th:text="${course.courseName}">Intro to
Programming</td>
                            <td th:text="${course.lecturerName}">Dr. Smith</td>
                            <td th:text="${course.credits}">3</td>
                            <td>
                                <span class="badge bg-success"
                                      th:if="${course.enrollmentStatus ==
'APPROVED'}">Approved</span>
                                <span class="badge bg-warning"
                                      th:if="${course.enrollmentStatus ==
'PENDING'}">Pending</span>
                            </td>
                        </tr>
                    </tbody>
                </table>
            </div>
        </div>

        <!-- Attendance Chart -->
        <div class="card mt-4">
            <div class="card-header">
                <h5>Attendance Overview</h5>
            </div>
            <div class="card-body">
                <canvas id="attendanceChart"></canvas>
            </div>
        </div>
    </div>
</div>

<script th:inline="javascript">
    // Attendance chart
    const ctx = document.getElementById('attendanceChart').getContext('2d');
    const attendanceData = [[${attendanceData}]];

    new Chart(ctx, {
        type: 'bar',
        data: {
            labels: attendanceData.map(d => d.courseName),
            datasets: [{
```

```
                label: 'Attendance Percentage',
                data: attendanceData.map(d => d.percentage),
                backgroundColor: 'rgba(54, 162, 235, 0.2)',
                borderColor: 'rgba(54, 162, 235, 1)',
                borderWidth: 1
            }]
        },
        options: {
            scales: {
                y: {
                    beginAtZero: true,
                    max: 100
                }
            }
        }
    });
</script>
```

11.3 Form Implementation

Course Enrollment Form:

```
<div layout:decorate="~{layout/main}">
    <div layout:fragment="content">
        <h1 class="h2">Enroll in Course</h1>

        <div class="card">
            <div class="card-body">
                <form th:action="@{/student/enroll}" method="post"
th:object="${enrollmentRequest}">
                    <div class="mb-3">
                        <label for="courseId" class="form-label">Select
Course</label>
                        <select class="form-select" id="courseId" th:field="*
{courseId}" required>
                            <option value="">-- Select a Course --</option>
                            <option th:each="course : ${availableCourses}"
                                    th:value="${course.id}"
                                    th:text="${course.courseCode + ' - ' +
course.courseName}">
                            </option>
                        </select>
                        <div class="invalid-feedback"
th:if="${#fields.hasErrors('courseId')}"
                             th:errors="*{courseId}"></div>
                    </div>

                    <div class="mb-3">
                        <label class="form-label">Course Details</label>
                        <div id="courseDetails" class="alert alert-info"
style="display:none;">
```

```
                            <!-- Course details will be loaded here via AJAX -->
                        </div>
                    </div>

                    <div class="form-check mb-3">
                        <input class="form-check-input" type="checkbox"
id="agreeTerms" required>
                        <label class="form-check-label" for="agreeTerms">
                            I agree to the enrollment terms and conditions
                        </label>
                    </div>

                    <button type="submit" class="btn btn-primary">Enroll</button>
                    <a th:href="@{/student/courses}" class="btn btn-
secondary">Cancel</a>
                </form>
            </div>
        </div>
    </div>
</div>

<script>
    $('#courseId').change(function() {
        var courseId = $(this).val();
        if (courseId) {
            $.ajax({
                url: '/api/courses/' + courseId,
                method: 'GET',
                success: function(course) {
                    var details = '<strong>Lecturer:</strong> ' +
course.lecturerName + '<br>' +
                                  '<strong>Credits:</strong> ' + course.credits +
'<br>' +
                                  '<strong>Max Students:</strong> ' +
course.maxStudents + '<br>' +
                                  '<strong>Available Seats:</strong> ' +
course.availableSeats;
                    $('#courseDetails').html(details).show();
                }
            });
        } else {
            $('#courseDetails').hide();
        }
    });
</script>
```

11.4 AJAX Implementation

Dynamic Course Loading:

```
// Load courses by department
function loadCoursesByDepartment(departmentId) {
    $.ajax({
        url: '/api/courses',
        method: 'GET',
        data: { departmentId: departmentId },
        success: function(courses) {
            var options = '<option value="">-- Select a Course --</option>';
            courses.forEach(function(course) {
                options += '<option value="' + course.id + '">' +
                           course.courseCode + ' - ' + course.courseName +
'</option>';
            });
            $('#courseId').html(options);
        },
        error: function(xhr, status, error) {
            console.error('Error loading courses:', error);
            alert('Failed to load courses');
        }
    });
}

// Real-time enrollment validation
function validateEnrollment(studentId, courseId) {
    return $.ajax({
        url: '/api/enrollments/validate',
        method: 'POST',
        contentType: 'application/json',
        data: JSON.stringify({ studentId: studentId, courseId: courseId })
    });
}

// Submit enrollment
$('#enrollmentForm').submit(function(e) {
    e.preventDefault();

    var formData = {
        studentId: $('#studentId').val(),
        courseId: $('#courseId').val()
    };

    $.ajax({
        url: '/api/enrollments',
        method: 'POST',
        contentType: 'application/json',
        data: JSON.stringify(formData),
        success: function(response) {
            alert('Enrollment request submitted successfully!');
            window.location.href = '/student/enrollments';
        },
        error: function(xhr, status, error) {
            var message = xhr.responseJSON ? xhr.responseJSON.message :
'Enrollment failed';
```

```
            alert(message);
        }
    });
});
```

11.5 Responsive Design

Custom CSS (style.css):

```
/* Layout */
body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background-color: #f5f5f5;
}

.sidebar {
    position: fixed;
    top: 56px;
    bottom: 0;
    left: 0;
    z-index: 100;
    padding: 48px 0 0;
    box-shadow: inset -1px 0 0 rgba(0, 0, 0, .1);
    background-color: #f8f9fa;
}

.sidebar-sticky {
    position: relative;
    top: 0;
    height: calc(100vh - 48px);
    padding-top: .5rem;
    overflow-x: hidden;
    overflow-y: auto;
}

/* Dashboard Cards */
.dashboard-card {
    border-radius: 10px;
    box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
    transition: transform 0.3s ease;
}

.dashboard-card:hover {
    transform: translateY(-5px);
    box-shadow: 0 6px 12px rgba(0, 0, 0, 0.15);
}

/* Tables */
.table-responsive {
    border-radius: 8px;
    overflow: hidden;
```

```
}

.table thead th {
    background-color: #007bff;
    color: white;
    font-weight: 600;
    border: none;
}

.table tbody tr:hover {
    background-color: #f1f3f5;
}

/* Buttons */
.btn {
    border-radius: 6px;
    padding: 8px 20px;
    font-weight: 500;
    transition: all 0.3s ease;
}

.btn:hover {
    transform: translateY(-2px);
    box-shadow: 0 4px 8px rgba(0, 0, 0, 0.2);
}

/* Forms */
.form-control, .form-select {
    border-radius: 6px;
    border: 1px solid #ced4da;
    padding: 10px 15px;
}

.form-control:focus, .form-select:focus {
    border-color: #007bff;
    box-shadow: 0 0 0 0.2rem rgba(0, 123, 255, 0.25);
}

/* Navigation */
.navbar {
    box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
}

.nav-link {
    color: #495057;
    padding: 10px 15px;
    border-radius: 6px;
    transition: background-color 0.3s ease;
}

.nav-link:hover {
    background-color: #e9ecef;
}
```

```
.nav-link.active {
    background-color: #007bff;
    color: white;
}

/* Badges */
.badge {
    padding: 6px 12px;
    border-radius: 20px;
    font-weight: 500;
}

/* Mobile Responsive */
@media (max-width: 768px) {
    .sidebar {
        position: relative;
        top: 0;
        height: auto;
    }

    .dashboard-card {
        margin-bottom: 15px;
    }

    .table {
        font-size: 0.9rem;
    }
}

/* Charts */
canvas {
    max-height: 400px;
}

/* Profile Picture */
.profile-picture {
    width: 150px;
    height: 150px;
    border-radius: 50%;
    object-fit: cover;
    border: 4px solid #007bff;
}

/* Loading Spinner */
.spinner-overlay {
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background: rgba(0, 0, 0, 0.5);
    display: flex;
    justify-content: center;
    align-items: center;
```

```
    z-index: 9999;
}
```

---

12. DATABASE IMPLEMENTATION

12.1 Flyway Migrations

Migration V1: Base Tables

```sql
-- V1__Create_base_tables.sql
CREATE TABLE IF NOT EXISTS users (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    phone_number VARCHAR(20),
    profile_picture VARCHAR(255),
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    INDEX idx_username (username),
    INDEX idx_email (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS roles (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL UNIQUE,
    description VARCHAR(255)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS user_roles (
    user_id BIGINT NOT NULL,
    role_id BIGINT NOT NULL,
    PRIMARY KEY (user_id, role_id),
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (role_id) REFERENCES roles(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS departments (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    department_name VARCHAR(100) NOT NULL UNIQUE,
    department_code VARCHAR(20) NOT NULL UNIQUE,
    description TEXT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS students (
    id BIGINT PRIMARY KEY,
    student_id VARCHAR(20) NOT NULL UNIQUE,
```

```
    major VARCHAR(100),
    year_level INT CHECK (year_level BETWEEN 1 AND 4),
    gpa DECIMAL(3,2) CHECK (gpa BETWEEN 0.00 AND 4.00),
    enrollment_date DATE,
    FOREIGN KEY (id) REFERENCES users(id) ON DELETE CASCADE,
    INDEX idx_student_id (student_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS lecturers (
    id BIGINT PRIMARY KEY,
    employee_id VARCHAR(20) NOT NULL UNIQUE,
    specialization VARCHAR(100),
    office_room VARCHAR(50),
    department_id BIGINT,
    FOREIGN KEY (id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (department_id) REFERENCES departments(id) ON DELETE SET NULL,
    INDEX idx_employee_id (employee_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS admins (
    id BIGINT PRIMARY KEY,
    admin_id VARCHAR(20) NOT NULL UNIQUE,
    department VARCHAR(100),
    admin_level VARCHAR(50),
    FOREIGN KEY (id) REFERENCES users(id) ON DELETE CASCADE,
    INDEX idx_admin_id (admin_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS semesters (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    semester_name VARCHAR(50) NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    is_active BOOLEAN DEFAULT TRUE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS courses (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    course_code VARCHAR(20) NOT NULL UNIQUE,
    course_name VARCHAR(100) NOT NULL,
    description TEXT,
    credits INT NOT NULL CHECK (credits > 0),
    max_students INT NOT NULL CHECK (max_students > 0),
    status ENUM('ACTIVE', 'INACTIVE', 'ARCHIVED') DEFAULT 'ACTIVE',
    lecturer_id BIGINT,
    department_id BIGINT,
    semester_id BIGINT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (lecturer_id) REFERENCES lecturers(id) ON DELETE SET NULL,
    FOREIGN KEY (department_id) REFERENCES departments(id) ON DELETE SET NULL,
    FOREIGN KEY (semester_id) REFERENCES semesters(id) ON DELETE SET NULL,
    INDEX idx_course_code (course_code),
    INDEX idx_status (status)
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS enrollments (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    student_id BIGINT NOT NULL,
    course_id BIGINT NOT NULL,
    status ENUM('PENDING', 'APPROVED', 'REJECTED', 'DROPPED') DEFAULT 'PENDING',
    enrolled_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    approved_at TIMESTAMP NULL,
    grade VARCHAR(5),
    FOREIGN KEY (student_id) REFERENCES students(id) ON DELETE CASCADE,
    FOREIGN KEY (course_id) REFERENCES courses(id) ON DELETE CASCADE,
    UNIQUE KEY unique_enrollment (student_id, course_id),
    INDEX idx_status (status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS classrooms (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    room_number VARCHAR(50) NOT NULL,
    building VARCHAR(100) NOT NULL,
    capacity INT NOT NULL CHECK (capacity > 0),
    has_projector BOOLEAN DEFAULT FALSE,
    has_whiteboard BOOLEAN DEFAULT TRUE,
    is_available BOOLEAN DEFAULT TRUE,
    UNIQUE KEY unique_classroom (room_number, building)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS time_slots (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    start_time TIME NOT NULL,
    end_time TIME NOT NULL,
    day_of_week ENUM('MONDAY', 'TUESDAY', 'WEDNESDAY', 'THURSDAY', 'FRIDAY',
'SATURDAY', 'SUNDAY') NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS schedules (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    course_id BIGINT NOT NULL,
    classroom_id BIGINT NOT NULL,
    time_slot_id BIGINT NOT NULL,
    effective_date DATE NOT NULL,
    expiry_date DATE,
    FOREIGN KEY (course_id) REFERENCES courses(id) ON DELETE CASCADE,
    FOREIGN KEY (classroom_id) REFERENCES classrooms(id) ON DELETE CASCADE,
    FOREIGN KEY (time_slot_id) REFERENCES time_slots(id) ON DELETE CASCADE,
    INDEX idx_course_schedule (course_id, effective_date)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

12.2 Data Seeding

Migration V2: Initial Data

```sql
-- V2__Insert_initial_data.sql

-- Insert default roles
INSERT INTO roles (name, description) VALUES
('ROLE_ADMIN', 'System Administrator with full access'),
('ROLE_LECTURER', 'Lecturer with course management access'),
('ROLE_STUDENT', 'Student with enrollment access');

-- Insert departments
INSERT INTO departments (department_name, department_code, description) VALUES
('Computer Science', 'CS', 'Department of Computer Science and Information
Technology'),
('Mathematics', 'MATH', 'Department of Mathematics and Statistics'),
('Physics', 'PHYS', 'Department of Physics and Astronomy'),
('Engineering', 'ENG', 'Department of Engineering'),
('Business', 'BUS', 'Department of Business Administration');

-- Insert default admin user (password: admin123)
INSERT INTO users (username, email, password, first_name, last_name, is_active)
VALUES
('admin', 'admin@university.edu',
'$2a$10$6YdXcN5z8ZXn3cC3w5ELUeGGP0IRZD0RvHnGGTxLi2J5c1T.Q7Py2', 'System',
'Administrator', TRUE);

SET @admin_user_id = LAST_INSERT_ID();

INSERT INTO admins (id, admin_id, department, admin_level) VALUES
(@admin_user_id, 'ADM001', 'IT', 'SUPER_ADMIN');

INSERT INTO user_roles (user_id, role_id)
SELECT @admin_user_id, id FROM roles WHERE name = 'ROLE_ADMIN';

-- Insert active semester
INSERT INTO semesters (semester_name, start_date, end_date, is_active) VALUES
('Spring 2026', '2026-01-15', '2026-05-15', TRUE),
('Fall 2025', '2025-09-01', '2025-12-20', FALSE);

-- Insert sample classrooms
INSERT INTO classrooms (room_number, building, capacity, has_projector,
has_whiteboard, is_available) VALUES
('101', 'Science Building', 40, TRUE, TRUE, TRUE),
('102', 'Science Building', 35, TRUE, TRUE, TRUE),
('201', 'Engineering Building', 50, TRUE, TRUE, TRUE),
('301', 'Business Building', 30, TRUE, FALSE, TRUE),
('Lab-A', 'Computer Lab', 25, FALSE, TRUE, TRUE);

-- Insert time slots
INSERT INTO time_slots (start_time, end_time, day_of_week) VALUES
('08:00:00', '09:30:00', 'MONDAY'),
('08:00:00', '09:30:00', 'WEDNESDAY'),
('10:00:00', '11:30:00', 'TUESDAY'),
('10:00:00', '11:30:00', 'THURSDAY'),
('13:00:00', '14:30:00', 'MONDAY'),
```

```
('13:00:00', '14:30:00', 'WEDNESDAY'),
('15:00:00', '16:30:00', 'TUESDAY'),
('15:00:00', '16:30:00', 'THURSDAY');
```

12.3 Query Optimization

Indexed Queries:

1. **Find Student Enrollments:**

```
SELECT e.*, c.course_code, c.course_name
FROM enrollments e
INNER JOIN courses c ON e.course_id = c.id
WHERE e.student_id = ? AND e.status = 'APPROVED'
ORDER BY e.enrolled_at DESC;
```

2. **Course Availability Check:**

```
SELECT c.*,
       c.max_students - COUNT(e.id) as available_seats
FROM courses c
LEFT JOIN enrollments e ON c.id = e.course_id AND e.status = 'APPROVED'
WHERE c.status = 'ACTIVE'
GROUP BY c.id
HAVING available_seats > 0;
```

3. **Schedule Conflict Detection:**

```
SELECT s1.*
FROM schedules s1
INNER JOIN schedules s2 ON s1.classroom_id = s2.classroom_id
    AND s1.time_slot_id = s2.time_slot_id
    AND s1.id != s2.id
WHERE s1.course_id = ?
    AND s1.effective_date <= CURRENT_DATE
    AND (s1.expiry_date IS NULL OR s1.expiry_date >= CURRENT_DATE);
```

---

13. SECURITY IMPLEMENTATION

13.1 Authentication Implementation

Custom UserDetailsService:

```
@Service
public class CustomUserDetailsService implements UserDetailsService {

    private final UserRepository userRepository;

    @Override
    @Transactional(readOnly = true)
    public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException {
        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("User not found: " +
username));

        if (!user.getIsActive()) {
            throw new DisabledException("User account is disabled");
        }

        Set<GrantedAuthority> authorities = user.getRoles().stream()
            .map(role -> new SimpleGrantedAuthority(role.getName()))
            .collect(Collectors.toSet());

        return new org.springframework.security.core.userdetails.User(
            user.getUsername(),
            user.getPassword(),
            user.getIsActive(),
            true, // accountNonExpired
            true, // credentialsNonExpired
            true, // accountNonLocked
            authorities
        );
    }
}
```

Authentication Success Handler:

```
@Component
public class CustomAuthenticationSuccessHandler implements
AuthenticationSuccessHandler {

    private final LoginHistoryRepository loginHistoryRepository;
    private final UserRepository userRepository;

    @Override
    public void onAuthenticationSuccess(HttpServletRequest request,
                                        HttpServletResponse response,
                                        Authentication authentication) throws
IOException {
        // Log successful login
        String username = authentication.getName();
        User user = userRepository.findByUsername(username).orElse(null);
```

```
        if (user != null) {
            LoginHistory loginHistory = new LoginHistory();
            loginHistory.setUser(user);
            loginHistory.setLoginTime(LocalDateTime.now());
            loginHistory.setIpAddress(request.getRemoteAddr());
            loginHistory.setSuccess(true);
            loginHistoryRepository.save(loginHistory);
        }

        // Redirect based on role
        Collection<? extends GrantedAuthority> authorities =
authentication.getAuthorities();

        String redirectUrl = "/";
        for (GrantedAuthority authority : authorities) {
            if (authority.getAuthority().equals("ROLE_ADMIN")) {
                redirectUrl = "/admin/dashboard";
                break;
            } else if (authority.getAuthority().equals("ROLE_LECTURER")) {
                redirectUrl = "/lecturer/dashboard";
                break;
            } else if (authority.getAuthority().equals("ROLE_STUDENT")) {
                redirectUrl = "/student/dashboard";
                break;
            }
        }

        response.sendRedirect(redirectUrl);
    }
}
```

13.2 OAuth2 Implementation

OAuth2 Login Success Handler:

```
@Component
public class OAuth2LoginSuccessHandler implements AuthenticationSuccessHandler {

    private final UserService userService;
    private final RoleRepository roleRepository;

    @Override
    public void onAuthenticationSuccess(HttpServletRequest request,
                                        HttpServletResponse response,
                                        Authentication authentication) throws
IOException {
        OAuth2User oauth2User = (OAuth2User) authentication.getPrincipal();

        String email = oauth2User.getAttribute("email");
        String name = oauth2User.getAttribute("name");
```

```
        // Check if user exists
        User user = userService.findByEmail(email).orElse(null);

        if (user == null) {
            // Create new user
            user = new User();
            user.setEmail(email);
            user.setUsername(email);
            user.setFirstName(name.split(" ")[0]);
            user.setLastName(name.split(" ").length > 1 ? name.split(" ")[1] :
"");
            user.setIsActive(true);
            user.setPassword(""); // OAuth users don't have passwords

            // Assign default STUDENT role
            Role studentRole = roleRepository.findByName("ROLE_STUDENT")
                .orElseThrow(() -> new RuntimeException("Student role not
found"));
            user.setRoles(Set.of(studentRole));

            userService.save(user);
        }

        response.sendRedirect("/student/dashboard");
    }
}
```

13.3 Password Encryption

Password Encoder Configuration:

```
@Configuration
public class PasswordConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(10); // Strength 10
    }
}
```

Password Change Service:

```
@Service
public class PasswordService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    public void changePassword(Long userId, String oldPassword, String
```

```
newPassword) {
        User user = userRepository.findById(userId)
            .orElseThrow(() -> new ResourceNotFoundException("User not found"));

        // Verify old password
        if (!passwordEncoder.matches(oldPassword, user.getPassword())) {
            throw new IllegalArgumentException("Current password is incorrect");
        }

        // Validate new password
        validatePassword(newPassword);

        // Update password
        user.setPassword(passwordEncoder.encode(newPassword));
        userRepository.save(user);
    }

    private void validatePassword(String password) {
        if (password.length() < 8) {
            throw new IllegalArgumentException("Password must be at least 8
characters long");
        }

        if (!password.matches(".*[A-Z].*")) {
            throw new IllegalArgumentException("Password must contain at least one
uppercase letter");
        }

        if (!password.matches(".*[a-z].*")) {
            throw new IllegalArgumentException("Password must contain at least one
lowercase letter");
        }

        if (!password.matches(".*\\d.*")) {
            throw new IllegalArgumentException("Password must contain at least one
digit");
        }
    }
}
```

13.4 Session Management

Session Service Implementation:

```
@Service
public class SessionServiceImpl implements SessionService {

    private final UserSessionRepository sessionRepository;
    private final UserRepository userRepository;

    @Override
```

```
    public void createSession(String username, String sessionToken,
HttpServletRequest request) {
        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new ResourceNotFoundException("User not found"));

        UserSession session = new UserSession();
        session.setUser(user);
        session.setSessionToken(sessionToken);
        session.setIpAddress(request.getRemoteAddr());
        session.setUserAgent(request.getHeader("User-Agent"));
        session.setCreatedAt(LocalDateTime.now());
        session.setExpiresAt(LocalDateTime.now().plusMinutes(30));

        sessionRepository.save(session);
    }

    @Override
    public void invalidateSession(String sessionToken) {
        sessionRepository.deleteBySessionToken(sessionToken);
    }

    @Override
    @Scheduled(fixedRate = 300000) // Run every 5 minutes
    public void cleanupExpiredSessions() {
        LocalDateTime now = LocalDateTime.now();
        sessionRepository.deleteByExpiresAtBefore(now);
    }
}
```

---

14. INTEGRATION IMPLEMENTATION

14.1 Google Sheets Integration

Google Sheets Configuration:

```
@Configuration
public class GoogleSheetsConfig {

    @Value("${google.credentials.file}")
    private String credentialsFile;

    @Value("${google.application.name}")
    private String applicationName;

    @Bean
    public Sheets sheetsService() throws IOException, GeneralSecurityException {
        GoogleCredentials credentials = GoogleCredentials
            .fromStream(new FileInputStream(credentialsFile))
            .createScoped(Collections.singleton(SheetsScopes.SPREADSHEETS));
```

```
        HttpRequestInitializer requestInitializer = new
HttpCredentialsAdapter(credentials);

        return new Sheets.Builder(
            GoogleNetHttpTransport.newTrustedTransport(),
            GsonFactory.getDefaultInstance(),
            requestInitializer)
            .setApplicationName(applicationName)
            .build();
    }
}
```

Google Sheets Service Implementation:

```
@Service
public class GoogleSheetsServiceImpl implements GoogleSheetsService {

    private final Sheets sheetsService;
    private final CourseSheetRepository courseSheetRepository;

    @Override
    public String createCourseSheet(Long courseId, String courseName) throws
IOException {
        // Create new spreadsheet
        Spreadsheet spreadsheet = new Spreadsheet()
            .setProperties(new SpreadsheetProperties().setTitle(courseName + " -
Attendance"));

        spreadsheet = sheetsService.spreadsheets().create(spreadsheet).execute();
        String spreadsheetId = spreadsheet.getSpreadsheetId();

        // Set up headers
        List<List<Object>> headers = Arrays.asList(
            Arrays.asList("Date", "Student ID", "Student Name", "Status", "Notes")
        );

        ValueRange headerRange = new ValueRange()
            .setValues(headers);

        sheetsService.spreadsheets().values()
            .update(spreadsheetId, "Sheet1!A1:E1", headerRange)
            .setValueInputOption("RAW")
            .execute();

        // Format headers
        List<Request> requests = new ArrayList<>();
        requests.add(new Request()
            .setRepeatCell(new RepeatCellRequest()
                .setRange(new GridRange()
                    .setSheetId(0)
                    .setStartRowIndex(0)
                    .setEndRowIndex(1))
```

```
                .setCell(new CellData()
                    .setUserEnteredFormat(new CellFormat()
                        .setBackgroundColor(new Color()
                            .setRed(0.2f)
                            .setGreen(0.6f)
                            .setBlue(0.86f))
                        .setTextFormat(new TextFormat()
                            .setBold(true)
                            .setForegroundColor(new Color()
                                .setRed(1f)
                                .setGreen(1f)
                                .setBlue(1f)))))
                .setFields("userEnteredFormat(backgroundColor,textFormat)")));

        BatchUpdateSpreadsheetRequest batchRequest = new
BatchUpdateSpreadsheetRequest()
            .setRequests(requests);

        sheetsService.spreadsheets().batchUpdate(spreadsheetId,
batchRequest).execute();

        // Save to database
        CourseSheet courseSheet = new CourseSheet();
        courseSheet.setCourseId(courseId);
        courseSheet.setSpreadsheetId(spreadsheetId);
        courseSheet.setSheetName("Sheet1");
        courseSheetRepository.save(courseSheet);

        return spreadsheetId;
    }

    @Override
    public void syncAttendanceToSheet(Long courseId, List<Attendance>
attendanceList) throws IOException {
        CourseSheet courseSheet = courseSheetRepository.findByCourseId(courseId)
            .orElseThrow(() -> new ResourceNotFoundException("Course sheet not
found"));

        // Prepare data
        List<List<Object>> values = new ArrayList<>();
        for (Attendance attendance : attendanceList) {
            values.add(Arrays.asList(
                attendance.getAttendanceDate().toString(),
                attendance.getStudent().getStudentId(),
                attendance.getStudent().getFirstName() + " " +
attendance.getStudent().getLastName(),
                attendance.getStatus().toString(),
                attendance.getNotes() != null ? attendance.getNotes() : ""
            ));
        }

        ValueRange body = new ValueRange().setValues(values);

        sheetsService.spreadsheets().values()
```

```
            .append(courseSheet.getSpreadsheetId(),
                    courseSheet.getSheetName() + "!A2",
                    body)
            .setValueInputOption("RAW")
            .execute();
    }

    @Override
    public List<AttendanceDTO> readAttendanceFromSheet(Long courseId) throws
IOException {
        CourseSheet courseSheet = courseSheetRepository.findByCourseId(courseId)
            .orElseThrow(() -> new ResourceNotFoundException("Course sheet not
found"));

        ValueRange response = sheetsService.spreadsheets().values()
            .get(courseSheet.getSpreadsheetId(), courseSheet.getSheetName() +
"!A2:E")
            .execute();

        List<List<Object>> values = response.getValues();
        List<AttendanceDTO> attendanceList = new ArrayList<>();

        if (values != null && !values.isEmpty()) {
            for (List<Object> row : values) {
                AttendanceDTO dto = new AttendanceDTO();
                dto.setAttendanceDate(LocalDate.parse(row.get(0).toString()));
                dto.setStudentId(row.get(1).toString());
                dto.setStudentName(row.get(2).toString());
                dto.setStatus(row.get(3).toString());
                dto.setNotes(row.size() > 4 ? row.get(4).toString() : "");
                attendanceList.add(dto);
            }
        }

        return attendanceList;
    }
}
```

14.2 File Upload Service

File Storage Service:

```
@Service
public class FileStorageService {

    private final Path uploadPath;

    public FileStorageService(@Value("${file.upload.dir}") String uploadDir) {
        this.uploadPath = Paths.get(uploadDir).toAbsolutePath().normalize();
        try {
            Files.createDirectories(this.uploadPath);
```

```
        } catch (IOException ex) {
            throw new RuntimeException("Could not create upload directory", ex);
        }
    }

    public String storeFile(MultipartFile file) {
        String fileName = StringUtils.cleanPath(file.getOriginalFilename());

        try {
            // Validate file
            if (fileName.contains("..")) {
                throw new IllegalArgumentException("Invalid file path: " +
fileName);
            }

            // Generate unique filename
            String fileExtension = getFileExtension(fileName);
            String uniqueFileName = UUID.randomUUID().toString() + fileExtension;

            // Copy file to upload directory
            Path targetLocation = this.uploadPath.resolve(uniqueFileName);
            Files.copy(file.getInputStream(), targetLocation,
StandardCopyOption.REPLACE_EXISTING);

            return uniqueFileName;
        } catch (IOException ex) {
            throw new RuntimeException("Could not store file " + fileName, ex);
        }
    }

    public Resource loadFileAsResource(String fileName) {
        try {
            Path filePath = this.uploadPath.resolve(fileName).normalize();
            Resource resource = new UrlResource(filePath.toUri());

            if (resource.exists()) {
                return resource;
            } else {
                throw new ResourceNotFoundException("File not found: " +
fileName);
            }
        } catch (MalformedURLException ex) {
            throw new ResourceNotFoundException("File not found: " + fileName);
        }
    }

    public void deleteFile(String fileName) {
        try {
            Path filePath = this.uploadPath.resolve(fileName).normalize();
            Files.deleteIfExists(filePath);
        } catch (IOException ex) {
            throw new RuntimeException("Could not delete file: " + fileName);
        }
    }
```

```
    private String getFileExtension(String fileName) {
        int lastDotIndex = fileName.lastIndexOf('.');
        return lastDotIndex == -1 ? "" : fileName.substring(lastDotIndex);
    }
}
```

PART IV: FEATURES

15. ADMIN MODULE FEATURES

15.1 User Management

Features:

- ☑ Create, view, update, and delete users (Students, Lecturers, Admins)
- ☑ Activate/deactivate user accounts
- ☑ Reset user passwords
- ☑ Assign and manage user roles
- ☑ View user activity logs
- ☑ Search and filter users by role, status, department

Implementation Highlights:

```
@PreAuthorize("hasRole('ADMIN')")
@PostMapping("/users/create")
public String createUser(@Valid @ModelAttribute UserDTO userDTO, BindingResult
result) {
    if (result.hasErrors()) {
        return "admin/user-create";
    }

    // Create user based on role
    if (userDTO.getRole().equals("STUDENT")) {
        studentService.createStudent(convertToStudentDTO(userDTO));
    } else if (userDTO.getRole().equals("LECTURER")) {
        lecturerService.createLecturer(convertToLecturerDTO(userDTO));
    } else if (userDTO.getRole().equals("ADMIN")) {
        adminService.createAdmin(convertToAdminDTO(userDTO));
    }

    return "redirect:/admin/users";
}
```

15.2 Course Management

Features:

- ☑ Create new courses with full details
- ☑ Edit existing course information
- ☑ Assign lecturers to courses
- ☑ Set course capacity and prerequisites
- ☑ Activate/deactivate courses
- ☑ Archive old courses
- ☑ View course enrollment statistics

Course Creation Flow:

1. Admin enters course details (code, name, credits, capacity)
2. System validates course code uniqueness
3. Admin assigns lecturer and department
4. Admin sets semester and schedule
5. Course status set to ACTIVE
6. Students can now enroll

15.3 Enrollment Management

Features:

- ☑ View all enrollment requests
- ☑ Approve or reject enrollments
- ☑ View enrollment statistics by course
- ☑ Generate enrollment reports
- ☑ Bulk enrollment operations
- ☑ Enrollment waitlist management

Approval Process:

```
@PreAuthorize("hasRole('ADMIN')")
@PostMapping("/enrollments/{id}/approve")
public String approveEnrollment(@PathVariable Long id) {
    Enrollment enrollment = enrollmentService.getById(id);

    // Check course capacity
    long currentEnrollments =
enrollmentService.countApprovedByCourse(enrollment.getCourse().getId());
    if (currentEnrollments >= enrollment.getCourse().getMaxStudents()) {
        throw new IllegalStateException("Course is full");
    }

    enrollment.setStatus(EnrollmentStatus.APPROVED);
    enrollment.setApprovedAt(LocalDateTime.now());
    enrollmentService.update(enrollment);

    return "redirect:/admin/enrollments";
}
```

15.4 Schedule Management

Features:

- ☑ Create course schedules
- ☑ Assign classrooms to courses
- ☑ Set time slots and days
- ☑ Detect scheduling conflicts
- ☑ View schedule calendar
- ☑ Generate timetables
- ☑ Classroom availability checking

Conflict Detection:

```
public boolean hasScheduleConflict(Schedule newSchedule) {
    List<Schedule> existingSchedules = scheduleRepository
        .findByClassroomIdAndTimeSlotId(
            newSchedule.getClassroom().getId(),
            newSchedule.getTimeSlot().getId()
        );

    for (Schedule schedule : existingSchedules) {
        if (isDateRangeOverlapping(newSchedule, schedule)) {
            return true;
        }
    }

    return false;
}
```

15.5 Reports and Analytics

Features:

- ☑ Dashboard with key statistics
- ☑ Enrollment trends report
- ☑ Course popularity analysis
- ☑ Attendance reports
- ☑ User activity reports
- ☑ Department-wise statistics
- ☑ Export reports to PDF/Excel

Dashboard Statistics:

- Total students, lecturers, courses
- Active enrollments
- Pending approval count
- Course capacity utilization

- Attendance rates
- Recent activities

15.6 System Configuration

Features:

- ☑ Semester management
- ☑ Department management
- ☑ Classroom management
- ☑ Time slot configuration
- ☑ System settings
- ☑ Email templates
- ☑ Backup and restore

---

16. LECTURER MODULE FEATURES

16.1 Course Management

Features:

- ☑ View assigned courses
- ☑ Update course descriptions
- ☑ View enrolled students
- ☑ Manage course materials
- ☑ Set course policies
- ☑ View course schedule

My Courses Dashboard:

```
<div class="card">
    <div class="card-header">
        <h5>My Courses</h5>
    </div>
    <div class="card-body">
        <div class="row">
            <div class="col-md-4" th:each="course : ${myCourses}">
                <div class="card mb-3">
                    <div class="card-body">
                        <h5 th:text="${course.courseName}">Course Name</h5>
                        <p class="text-muted"
th:text="${course.courseCode}">CS101</p>
                        <p>
                            <strong>Enrolled:</strong>
                            <span th:text="${course.enrolledCount} + '/' +
${course.maxStudents}">25/40</span>
                        </p>
                        <a th:href="@{/lecturer/courses/{id}(id=${course.id})}"
                           class="btn btn-primary btn-sm">View Details</a>
                    </div>
```

```
                </div>
            </div>
        </div>
    </div>
</div>
```

16.2 Attendance Management

Features:

- ☑ Mark attendance for classes
- ☑ View attendance history
- ☑ Update attendance records
- ☑ Sync with Google Sheets
- ☑ Generate attendance reports
- ☑ Export attendance data
- ☑ View student attendance patterns

Attendance Marking Interface:

```
@PreAuthorize("hasRole('LECTURER')")
@PostMapping("/attendance/mark")
public String markAttendance(@RequestParam Long courseId,
                             @RequestParam LocalDate date,
                             @RequestParam Map<String, String> attendanceData) {
    List<Attendance> attendanceList = new ArrayList<>();

    for (Map.Entry<String, String> entry : attendanceData.entrySet()) {
        if (entry.getKey().startsWith("student_")) {
            Long studentId = Long.parseLong(entry.getKey().substring(8));
            String status = entry.getValue();

            Attendance attendance = new Attendance();
            attendance.setStudentId(studentId);
            attendance.setCourseId(courseId);
            attendance.setAttendanceDate(date);
            attendance.setStatus(AttendanceStatus.valueOf(status));

            attendanceList.add(attendance);
        }
    }

    attendanceService.saveAll(attendanceList);

    // Sync to Google Sheets
    googleSheetsService.syncAttendanceToSheet(courseId, attendanceList);

    return "redirect:/lecturer/attendance?courseId=" + courseId;
}
```

16.3 Student Management

Features:

- ☑ View enrolled students per course
- ☑ View student profiles
- ☑ View student attendance records
- ☑ View student grades (if applicable)
- ☑ Communication with students
- ☑ Export student lists

Enrolled Students View:

```
@GetMapping("/courses/{courseId}/students")
public String viewEnrolledStudents(@PathVariable Long courseId, Model model) {
    Course course = courseService.getCourseById(courseId);
    List<StudentDTO> students = enrollmentService.getEnrolledStudents(courseId);

    model.addAttribute("course", course);
    model.addAttribute("students", students);
    model.addAttribute("enrollmentCount", students.size());

    return "lecturer/course-students";
}
```

16.4 Schedule Management

Features:

- ☑ View personal teaching schedule
- ☑ View weekly timetable
- ☑ View classroom assignments
- ☑ Request schedule changes
- ☑ View conflicts and availability
- ☑ Export schedule to calendar

16.5 Profile Management

Features:

- ☑ View and edit profile information
- ☑ Update contact details
- ☑ Upload profile picture
- ☑ Set office hours
- ☑ Update specialization
- ☑ Change password

---

17. STUDENT MODULE FEATURES

17.1 Course Browsing and Enrollment

Features:

- ☑ Browse available courses
- ☑ Search courses by code, name, department
- ☑ Filter courses by semester, credits, lecturer
- ☑ View course details and descriptions
- ☑ Check course availability and seats
- ☑ Enroll in courses
- ☑ View prerequisites
- ☑ View course schedules

Course Browsing Interface:

```
<div class="card">
    <div class="card-header">
        <h5>Available Courses</h5>
        <div class="row mt-3">
            <div class="col-md-4">
                <input type="text" class="form-control" id="searchCourse"
                       placeholder="Search by code or name">
            </div>
            <div class="col-md-3">
                <select class="form-select" id="filterDepartment">
                    <option value="">All Departments</option>
                    <option th:each="dept : ${departments}"
                            th:value="${dept.id}"
                            th:text="${dept.departmentName}"></option>
                </select>
            </div>
            <div class="col-md-3">
                <select class="form-select" id="filterCredits">
                    <option value="">All Credits</option>
                    <option value="1">1 Credit</option>
                    <option value="2">2 Credits</option>
                    <option value="3">3 Credits</option>
                    <option value="4">4 Credits</option>
                </select>
            </div>
            <div class="col-md-2">
                <button class="btn btn-primary" onclick="applyFilters()">
                    <i class="bi bi-search"></i> Search
                </button>
            </div>
        </div>
    </div>
    <div class="card-body">
        <div class="table-responsive">
            <table class="table table-hover">
                <thead>
                    <tr>
```

```
                        <th>Code</th>
                        <th>Course Name</th>
                        <th>Lecturer</th>
                        <th>Credits</th>
                        <th>Available Seats</th>
                        <th>Action</th>
                    </tr>
                </thead>
                <tbody id="courseTableBody">
                    <tr th:each="course : ${courses}">
                        <td th:text="${course.courseCode}">CS101</td>
                        <td th:text="${course.courseName}">Introduction to
Programming</td>
                        <td th:text="${course.lecturerName}">Dr. Smith</td>
                        <td th:text="${course.credits}">3</td>
                        <td>
                            <span th:text="${course.availableSeats} + '/' +
${course.maxStudents}">15/40</span>
                        </td>
                        <td>
                            <button class="btn btn-sm btn-primary"
                                    th:onclick="'enrollCourse(' + ${course.id} +
')'">
                                Enroll
                            </button>
                            <a th:href="@{/student/courses/{id}(id=${course.id})}"
                               class="btn btn-sm btn-info">Details</a>
                        </td>
                    </tr>
                </tbody>
            </table>
        </div>
    </div>
</div>
```

17.2 My Enrollments

Features:

- ☑ View all enrolled courses
- ☑ Check enrollment status (Pending, Approved, Rejected)
- ☑ Drop courses (within allowed period)
- ☑ View enrollment history
- ☑ Track approval timeline
- ☑ View course schedules

Enrollment Status Tracking:

```
@GetMapping("/enrollments")
public String viewMyEnrollments(Model model, Authentication authentication) {
    String username = authentication.getName();
```

```
    Student student = studentService.getByUsername(username);

    List<EnrollmentDTO> enrollments =
enrollmentService.getByStudentId(student.getId());

    // Separate by status
    List<EnrollmentDTO> pending = enrollments.stream()
        .filter(e -> e.getStatus() == EnrollmentStatus.PENDING)
        .collect(Collectors.toList());

    List<EnrollmentDTO> approved = enrollments.stream()
        .filter(e -> e.getStatus() == EnrollmentStatus.APPROVED)
        .collect(Collectors.toList());

    model.addAttribute("pendingEnrollments", pending);
    model.addAttribute("approvedEnrollments", approved);
    model.addAttribute("totalCredits", calculateTotalCredits(approved));

    return "enrollment/my-enrollments";
}
```

17.3 Schedule and Timetable

Features:

- ☑ View weekly class schedule
- ☑ View monthly calendar
- ☑ View classroom locations
- ☑ View time slots
- ☑ Export schedule
- ☑ Set reminders
- ☑ View schedule conflicts

Weekly Timetable View:

```
<div class="card">
    <div class="card-header">
        <h5>My Weekly Schedule</h5>
    </div>
    <div class="card-body">
        <table class="table table-bordered">
            <thead>
                <tr>
                    <th>Time</th>
                    <th>Monday</th>
                    <th>Tuesday</th>
                    <th>Wednesday</th>
                    <th>Thursday</th>
                    <th>Friday</th>
                </tr>
            </thead>
```

```
            <tbody>
                <tr th:each="timeSlot : ${timeSlots}">
                    <td th:text="${timeSlot.startTime} + ' - ' +
${timeSlot.endTime}">08:00-09:30</td>
                    <td th:each="day : ${daysOfWeek}">
                        <div th:if="${schedule.hasClass(day, timeSlot)}"
                             class="schedule-cell"
                             th:classappend="${schedule.getClass(day,
timeSlot).courseCode}">
                            <strong th:text="${schedule.getClass(day,
timeSlot).courseCode}">CS101</strong><br>
                            <small th:text="${schedule.getClass(day,
timeSlot).courseName}">Intro to Programming</small><br>
                            <small th:text="${schedule.getClass(day,
timeSlot).classroom}">Room 101</small>
                        </div>
                    </td>
                </tr>
            </tbody>
        </table>
    </div>
</div>
```

17.4 Attendance Tracking

Features:

- ☑ View attendance records for each course
- ☑ Check attendance percentage
- ☑ View attendance history
- ☑ View attendance alerts (low attendance)
- ☑ Download attendance reports

17.5 Profile Management

Features:

- ☑ View personal information
- ☑ Update contact details
- ☑ Upload profile picture
- ☑ Update academic information
- ☑ Change password
- ☑ View enrollment history
- ☑ View GPA

---

18. COMMON FEATURES

18.1 Authentication

Features:

- ☑ Login with username/password
- ☑ Login with Google OAuth2
- ☑ Remember me functionality
- ☑ Forgot password
- ☑ Password reset
- ☑ Session management
- ☑ Logout

18.2 Dashboard

Role-Specific Dashboards:

Admin Dashboard:

- Total users statistics
- Course statistics
- Enrollment statistics
- Recent activities
- System health indicators
- Quick actions

Lecturer Dashboard:

- My courses summary
- Today's classes
- Recent attendance
- Pending tasks
- Quick links

Student Dashboard:

- Enrolled courses
- Total credits
- GPA display
- Attendance summary
- Upcoming classes
- Quick enrollment

18.3 Notifications

Features:

- ☑ Success messages
- ☑ Error messages
- ☑ Warning alerts
- ☑ Info notifications
- ☑ Toast notifications
- ☑ Email notifications (configurable)

18.4 Search and Filters

Features:

- ☑ Global search
- ☑ Course search
- ☑ User search
- ☑ Advanced filters
- ☑ Sort options
- ☑ Pagination

18.5 Export and Reports

Features:

- ☑ Export to PDF
- ☑ Export to Excel
- ☑ Export to CSV
- ☑ Print-friendly views
- ☑ Custom report generation

PART VI: METHODS AND FUNCTIONS EXPLANATION

---

23. SERVICE LAYER METHODS

23.1 Student Service Methods

StudentServiceImpl Class - Complete Method Breakdown:

createStudent(StudentDTO dto)

```
@Transactional
public Student createStudent(StudentDTO dto)
```

Purpose: Creates a new student account in the system

Input Parameters:

- `dto` - StudentDTO containing student information (studentId, username, email, firstName, lastName, etc.)

Process Flow:

1. **Validation Check:** Verifies if student ID already exists using `studentRepository.existsByStudentId()`
2. **Duplicate Prevention:** Throws `DuplicateResourceException` if student ID is already in use
3. **Entity Creation:** Creates new Student entity and sets all properties
4. **Password Encoding:** Encodes default password "student123" using BCrypt

5. **Department Assignment:** Links student to department if departmentId is provided
6. **Role Assignment:** Assigns ROLE_STUDENT from database
7. **Database Persistence:** Saves student entity to database
8. **Return:** Returns saved Student entity with generated ID

Error Handling:

- `DuplicateResourceException` - If student ID exists
- `ResourceNotFoundException` - If department or role not found

Transaction: @Transactional ensures all operations succeed or rollback

---

updateStudent(Long id, StudentDTO dto)

```
@Transactional
public Student updateStudent(Long id, StudentDTO dto)
```

Purpose: Updates existing student information

Input Parameters:

- `id` - Student's database ID
- `dto` - StudentDTO with updated information

Process Flow:

1. **Retrieval:** Fetches student by ID from database
2. **Existence Check:** Throws exception if student not found
3. **Property Update:** Updates email, name, phone, profile picture, major, year level
4. **Department Update:** Updates department if provided
5. **Save:** Persists changes to database
6. **Return:** Returns updated Student entity

Fields Updated:

- Email, first name, last name, phone number
- Profile picture path
- Major, year level
- Department association

---

deleteStudent(Long id)

```
@Transactional
public void deleteStudent(Long id)
```

Purpose: Deletes a student from the system

Input Parameters:

- `id` - Student's database ID

Process Flow:

1. **Existence Check:** Verifies student exists
2. **Cascade Delete:** Database CASCADE rules delete related enrollments, attendance
3. **Deletion:** Removes student record from database

Note: Uses database CASCADE constraints to maintain referential integrity

---

getStudentById(Long id)

```
public Optional<Student> getStudentById(Long id)
```

Purpose: Retrieves student by database ID

Return: Optional - Empty if not found, otherwise contains Student entity

Usage Pattern:

```
Optional<Student> student = studentService.getStudentById(1L);
if (student.isPresent()) {
    // Use student.get()
}
```

---

getStudentByStudentId(String studentId)

```
public Optional<Student> getStudentByStudentId(String studentId)
```

Purpose: Retrieves student by their unique student ID (e.g., "STU001")

Use Case: Login, search, verification operations

---

getStudentByUsername(String username)

```
public Optional<Student> getStudentByUsername(String username)
```

Purpose: Retrieves student by username for authentication

Used By: Spring Security, Login controllers

---

getAllStudents()

```
public List<Student> getAllStudents()
```

Purpose: Retrieves all students from database

Returns: Complete list of all Student entities

Use Case: Admin user management, reports, statistics

---

getStudentsByDepartment(Long departmentId)

```
public List<Student> getStudentsByDepartment(Long departmentId)
```

Purpose: Filters students by department

Use Case: Department-specific reports, course planning

---

getStudentsByYearLevel(Integer yearLevel)

```
public List<Student> getStudentsByYearLevel(Integer yearLevel)
```

Purpose: Filters students by year (1-4)

Use Case: Year-specific operations, statistics

---

updateGPA(Long studentId, Double gpa)

```
@Transactional
public void updateGPA(Long studentId, Double gpa)
```

Purpose: Updates student's GPA

Input Parameters:

- `studentId` - Student's database ID

- `gpa` - New GPA value (0.00 - 4.00)

Process Flow:

1. Fetch student by ID
2. Update GPA field
3. Save to database

Use Case: Grade processing, semester end updates

---

convertToDTO(Student student)

```
public StudentDTO convertToDTO(Student student)
```

Purpose: Converts Student entity to Data Transfer Object

Why Needed:

- Prevents exposing sensitive data (password)
- Reduces data transfer size
- Provides clean API responses
- Enables custom field formatting

DTO vs Entity:

- DTO: For API/UI layer (no password, formatted data)
- Entity: For database persistence (complete data, relationships)

23.2 Enrollment Service Methods

EnrollmentServiceImpl Class - Complete Method Breakdown:

enrollStudent(Long studentId, Long courseId)

```
@Transactional
public Enrollment enrollStudent(Long studentId, Long courseId)
```

Purpose: Enrolls a student in a course

Process Flow:

1. **Validation:**
    - Fetches student entity (throws exception if not found)
    - Fetches course entity (throws exception if not found)
2. **Duplicate Check:**

- Verifies student not already enrolled using `existsByStudentIdAndCourseId()`
- Throws `DuplicateResourceException` if already enrolled

3. **Capacity Check:**
   - Calls `course.isFull()` to check available seats
   - Throws `RuntimeException` if course at capacity
4. **Enrollment Creation:**
   - Creates new Enrollment entity
   - Sets student and course references
   - Sets status to PENDING (awaits admin approval)
   - Records enrollment timestamp
5. **Persistence:**
   - Saves enrollment to database
   - Updates course enrollment count
6. **Return:** Saved Enrollment entity

Business Rules:

- Initial status is PENDING
- Requires admin approval to become ACTIVE
- Cannot enroll if course is full
- Cannot duplicate enrollment

---

updateEnrollmentStatus(Long enrollmentId, EnrollmentStatus status)

```
@Transactional
public Enrollment updateEnrollmentStatus(Long enrollmentId, EnrollmentStatus
status)
```

Purpose: Changes enrollment status (PENDING → APPROVED/REJECTED/DROPPED)

Status Values:

- `PENDING` - Awaiting approval
- `APPROVED` - Approved by admin, active enrollment
- `REJECTED` - Rejected by admin
- `DROPPED` - Student dropped the course

Process Flow:

1. Fetch enrollment by ID
2. Update status field
3. If approved, set enrollment timestamp

4. Save changes
5. Update course enrollment count

Use Case: Admin approval workflow, student course drop

---

updateGrade(Long enrollmentId, Double grade)

```
@Transactional
public Enrollment updateGrade(Long enrollmentId, Double grade)
```

Purpose: Records final grade for an enrollment

Input:

- `enrollmentId` - Enrollment record ID
- `grade` - Numeric grade value

Process Flow:

1. Fetch enrollment record
2. Set grade field
3. Save to database

Use Case: End of semester grade submission

---

dropEnrollment(Long enrollmentId)

```
@Transactional
public void dropEnrollment(Long enrollmentId)
```

Purpose: Allows student to drop a course

Process Flow:

1. Fetch enrollment record
2. Change status to DROPPED
3. Save changes
4. Update course enrollment count (frees up seat)

Business Rule: Dropped enrollments free up course capacity

---

getEnrollmentsByStudent(Long studentId)

```
public List<Enrollment> getEnrollmentsByStudent(Long studentId)
```

Purpose: Retrieves all enrollments for a student

Returns: List of Enrollment entities with eager-loaded course and lecturer data

Query Optimization: Uses JOIN FETCH to prevent N+1 queries:

```
@Query("SELECT e FROM Enrollment e " +
       "JOIN FETCH e.course c " +
       "JOIN FETCH c.lecturer " +
       "WHERE e.student.id = :studentId")
List<Enrollment> findByStudentIdWithCourseAndLecturer(@Param("studentId") Long
studentId);
```

Use Case: Student dashboard, my courses page

---

getEnrollmentsByCourse(Long courseId)

```
public List<Enrollment> getEnrollmentsByCourse(Long courseId)
```

Purpose: Retrieves all students enrolled in a course

Use Case: Lecturer viewing enrolled students, attendance marking

---

getEnrollmentsByStatus(EnrollmentStatus status)

```
public List<Enrollment> getEnrollmentsByStatus(EnrollmentStatus status)
```

Purpose: Filters enrollments by status

Use Case: Admin viewing pending approvals

---

isStudentEnrolled(Long studentId, Long courseId)

```
public boolean isStudentEnrolled(Long studentId, Long courseId)
```

Purpose: Quick check if student is enrolled in course

Returns: true/false

Use Case: Preventing duplicate enrollments, checking prerequisites

---

approveEnrollment(Long enrollmentId)

```
@Transactional
public Enrollment approveEnrollment(Long enrollmentId)
```

Purpose: Convenience method for admin to approve enrollment

Implementation: Calls `updateEnrollmentStatus(enrollmentId, APPROVED)`

---

rejectEnrollment(Long enrollmentId)

```
@Transactional
public Enrollment rejectEnrollment(Long enrollmentId)
```

Purpose: Convenience method for admin to reject enrollment

Implementation: Calls `updateEnrollmentStatus(enrollmentId, REJECTED)`

23.3 Attendance Service Methods

AttendanceServiceImpl Class - Complete Method Breakdown:

markAttendance(AttendanceDTO dto)

```
public Attendance markAttendance(AttendanceDTO dto)
```

Purpose: Records student attendance for a class session

Input Parameters (AttendanceDTO):

- `scheduleId` - Class schedule ID
- `studentId` - Student ID
- `attendanceDate` - Date of class
- `status` - Attendance status (PRESENT, ABSENT, LATE, EXCUSED)
- `notes` - Optional notes
- `markedBy` - Lecturer username

Process Flow:

1. **Validation:**
    - Fetch schedule entity (validates class exists)
    - Fetch student entity (validates student exists)

2. **Duplicate Check:**
    - Query: `findByScheduleIdAndStudentIdAndAttendanceDate()`
    - If exists: Update existing record
    - If not: Create new record
3. **Update or Create:**
    - **Update Path:** Modify status, notes, markedBy
    - **Create Path:** Create new Attendance entity with all fields
4. **Sync Flag:**
    - Set `googleSheetSynced = false` to trigger sync
5. **Persistence:**
    - Save to database
    - Return saved entity

Attendance Status Values:

- `PRESENT` - Student attended
- `ABSENT` - Student did not attend
- `LATE` - Student arrived late
- `EXCUSED` - Excused absence

Use Case: Daily attendance marking by lecturers

---

updateAttendance(Long id, AttendanceDTO dto)

```
public Attendance updateAttendance(Long id, AttendanceDTO dto)
```

Purpose: Corrects or updates existing attendance record

Process Flow:

1. Fetch attendance record by ID
2. Update status, notes, markedBy
3. Set googleSheetSynced = false (needs re-sync)
4. Save changes

Use Case: Correcting attendance errors, late submissions

---

getAttendanceBySchedule(Long scheduleId)

```
public List<Attendance> getAttendanceBySchedule(Long scheduleId)
```

Purpose: Gets all attendance records for a specific class schedule

Returns: List of all attendance records for that schedule

Use Case: Viewing attendance for a specific class session

---

getAttendanceByStudent(Long studentId)

```
public List<Attendance> getAttendanceByStudent(Long studentId)
```

Purpose: Gets complete attendance history for a student

Returns: All attendance records across all courses

Use Case: Student profile, attendance report

---

getAttendanceByCourse(Long courseId)

```
public List<Attendance> getAttendanceByCourse(Long courseId)
```

Purpose: Gets all attendance records for a course

Query: Joins through Schedule → Course relationship

Use Case: Course-wide attendance statistics

---

getAttendanceByScheduleAndDate(Long scheduleId, LocalDate date)

```
public List<Attendance> getAttendanceByScheduleAndDate(Long scheduleId, LocalDate
date)
```

Purpose: Gets attendance for specific date and schedule

Use Case: Viewing attendance for today's class

---

markBulkAttendance(List dtos)

```
public List<Attendance> markBulkAttendance(List<AttendanceDTO> dtos)
```

Purpose: Marks attendance for multiple students in one operation

Process Flow:

1. Iterates through DTO list
2. Calls `markAttendance()` for each
3. Collects all results
4. Returns list of saved records

Use Case: Lecturer marking attendance for entire class at once

Performance: Uses stream processing for efficiency:

```
return dtos.stream()
    .map(this::markAttendance)
    .collect(Collectors.toList());
```

---

convertToDTO(Attendance attendance)

```
public AttendanceDTO convertToDTO(Attendance attendance)
```

Purpose: Converts Attendance entity to DTO with additional display fields

Added Fields in DTO:

- `studentName` - Concatenated first + last name
- `studentNumber` - Student ID for display
- `courseName` - Course name from schedule
- `courseCode` - Course code from schedule

Use Case: API responses, UI rendering

---

getAttendanceSummaryForCourse(Long courseId)

```
public List<AttendanceDTO> getAttendanceSummaryForCourse(Long courseId)
```

Purpose: Gets attendance data formatted for reports

Process Flow:

1. Fetch all attendance records for course
2. Convert each to DTO (includes display fields)
3. Return formatted list

Use Case: Attendance reports, statistics, Google Sheets export

---

23.4 Schedule Service Methods

ScheduleServiceImpl Class - Complete Method Breakdown:

createSchedule(ScheduleDTO dto)

```
@Transactional
public Schedule createSchedule(ScheduleDTO dto)
```

Purpose: Creates a new class schedule

Input Parameters (ScheduleDTO):

- `courseId` - Course to schedule
- `classroomId` - Classroom assignment
- `dayOfWeek` - Day (MONDAY, TUESDAY, etc.)
- `startTime` - Class start time
- `endTime` - Class end time

Process Flow:

1. **Conflict Detection:**
    - Calls `conflictDetectionService.hasScheduleConflict(dto)`
    - Checks for classroom conflicts (same room, same time)
    - Checks for lecturer conflicts (same lecturer, same time)
    - Throws `ScheduleConflictException` if conflict found
2. **Entity Validation:**
    - Fetch course entity (validates exists)
    - Fetch classroom entity (validates exists)
3. **Schedule Creation:**
    - Create new Schedule entity
    - Set all relationships and time fields
    - Save to database
4. **Return:** Saved schedule entity

Conflict Types Detected:

- Classroom double-booking
- Lecturer teaching two classes simultaneously
- Student schedule conflicts (if implemented)

---

updateSchedule(Long id, ScheduleDTO dto)

```
@Transactional
public Schedule updateSchedule(Long id, ScheduleDTO dto)
```

Purpose: Updates existing schedule

Special Handling:

- Sets `dto.id = id` before conflict check
- Conflict detection excludes current schedule (can't conflict with itself)

Process Flow:

1. Fetch existing schedule
2. Run conflict check (excluding self)
3. Validate course and classroom
4. Update all fields
5. Save changes

---

deleteSchedule(Long id)

```
@Transactional
public void deleteSchedule(Long id)
```

Purpose: Removes a schedule

Impact: Related attendance records may need handling (depends on CASCADE rules)

---

getSchedulesByCourse(Long courseId)

```
public List<Schedule> getSchedulesByCourse(Long courseId)
```

Purpose: Gets all scheduled sessions for a course

Returns: List of schedules (multiple time slots per week)

Use Case: Course timetable display

---

getSchedulesByClassroom(Long classroomId)

```
public List<Schedule> getSchedulesByClassroom(Long classroomId)
```

Purpose: Gets all classes scheduled in a classroom

Use Case: Classroom utilization reports

---

getSchedulesByDay(DayOfWeek dayOfWeek)

```
public List<Schedule> getSchedulesByDay(DayOfWeek dayOfWeek)
```

Purpose: Gets all classes on a specific day

Use Case: Daily schedule view, today's classes

---

hasConflict(ScheduleDTO dto)

```
public boolean hasConflict(ScheduleDTO dto)
```

Purpose: Public method to check for scheduling conflicts

Returns: true if conflict exists, false if clear

Use Case: Pre-validation before showing schedule form

23.5 Course Service Methods

CourseServiceImpl Class - Key Methods:

createCourse(CourseDTO dto)

```
@Transactional
public Course createCourse(CourseDTO dto)
```

Purpose: Creates new course in system

Validation:

- Course code uniqueness
- Credits range (1-6)
- Max students > 0
- Lecturer exists
- Department exists

Sets:

- Default status: ACTIVE
- Initial enrollment count: 0

---

updateCourse(Long id, CourseDTO dto)

```
@Transactional
public Course updateCourse(Long id, CourseDTO dto)
```

Purpose: Updates course information

Allowed Updates:

- Course name, description
- Credits, max students
- Lecturer assignment
- Status (ACTIVE/INACTIVE/ARCHIVED)

Restricted:

- Course code cannot be changed (unique identifier)

---

updateEnrollmentCount(Long courseId)

```
@Transactional
public void updateEnrollmentCount(Long courseId)
```

Purpose: Recalculates enrolled student count

Process:

1. Count APPROVED enrollments
2. Update course's currentEnrollment field
3. Save course

Triggers:

- After new enrollment
- After enrollment approval
- After enrollment drop

Why Important: Enables quick capacity checks without counting every time

---

isFull(Course course)

```
public boolean isFull(Course course)
```

Purpose: Checks if course is at capacity

Logic: `course.getCurrentEnrollment() >= course.getMaxStudents()`

Use Case: Enrollment validation, UI indicators

---

24. CONTROLLER LAYER METHODS

24.1 Student Dashboard Controller Methods

StudentDashboardController Class - Complete Method Breakdown:

showDashboard(Model model, Authentication authentication)

```
@GetMapping("/student/dashboard")
public String showDashboard(Model model, Authentication authentication)
```

Purpose: Displays student's main dashboard page

HTTP Method: GET

Input Parameters:

- `model` - Spring Model for passing data to view
- `authentication` - Spring Security authentication object (auto-injected)

Process Flow:

1. **Get Current User:**
    - Extract username from `authentication.getName()`
    - Fetch Student entity from database
2. **Gather Dashboard Data:**
    - Enrolled courses count
    - Total credits enrolled
    - Current GPA
    - Attendance statistics
    - Recent activities
3. **Add to Model:**

```
model.addAttribute("student", student);
model.addAttribute("enrolledCourses", courses);
```

```
model.addAttribute("totalCredits", credits);
model.addAttribute("attendanceRate", rate);
```

4. **Return View:**
    - Returns "dashboard/student-dashboard"
    - Thymeleaf processes template with model data

Security: @PreAuthorize("hasRole('ROLE_STUDENT')") ensures only students access

View Template: `src/main/resources/templates/dashboard/student-dashboard.html`

---

showProfile(Model model, Authentication authentication)

```
@GetMapping("/student/profile")
public String showProfile(Model model, Authentication authentication)
```

Purpose: Displays student profile page

Data Loaded:

- Personal information
- Academic details
- Enrollment history
- Contact information

View: "student/profile"

---

editProfile(Model model, Authentication authentication)

```
@GetMapping("/student/profile/edit")
public String editProfile(Model model, Authentication authentication)
```

Purpose: Shows profile edit form

Form Binding:

- Loads current student data into StudentDTO
- Passes to view for form population

---

updateProfile(StudentDTO dto, BindingResult result, Authentication auth)

```
@PostMapping("/student/profile/update")
public String updateProfile(@Valid @ModelAttribute StudentDTO dto,
                            BindingResult result,
                            Authentication authentication,
                            RedirectAttributes redirectAttributes)
```

Purpose: Processes profile update form submission

Input Parameters:

- `@Valid` - Triggers JSR-303 validation on DTO
- `@ModelAttribute` - Binds form data to DTO
- `BindingResult` - Contains validation errors
- `RedirectAttributes` - For flash messages

Process Flow:

1. **Validation:**
    - Check for binding errors
    - If errors exist, return to edit form
2. **Authorization:**
    - Verify user is updating their own profile
    - Get student ID from authentication
3. **Update:**
    - Call `studentService.updateStudent(id, dto)`
    - Transaction ensures consistency
4. **Success Response:**
    - Add success message to flash attributes
    - Redirect to profile view (PRG pattern)

PRG Pattern (Post-Redirect-Get):

- Prevents form resubmission on refresh
- Flow: POST → Redirect → GET

Example:

```
if (result.hasErrors()) {
    return "student/edit-profile"; // Return to form
}

studentService.updateProfile(dto);
redirectAttributes.addFlashAttribute("success", "Profile updated!");
return "redirect:/student/profile"; // Redirect
```

---

uploadProfilePicture(MultipartFile file, Authentication auth)

```
@PostMapping("/student/profile/upload-picture")
public String uploadProfilePicture(@RequestParam("file") MultipartFile file,
                                   Authentication authentication,
                                   RedirectAttributes redirectAttributes)
```

Purpose: Handles profile picture file upload

Process Flow:

1. **File Validation:**
    - Check file is not empty
    - Validate file type (jpg, png, jpeg)
    - Check file size (< 10MB)
2. **Store File:**
    - Call `fileStorageService.storeFile(file)`
    - Returns unique filename
3. **Update Student:**
    - Save filename to student's profilePicture field
    - Database stores path, not binary data
4. **Response:**
    - Success message via flash attributes
    - Redirect to profile

File Storage:

- Files stored in `uploads/profiles/` directory
- Filename format: `UUID.extension`
- Example: `a869ec64-7115-4e0c-8214-268945e07bcb.jpeg`

---

viewMySchedule(Model model, Authentication authentication)

```
@GetMapping("/student/schedule")
public String viewMySchedule(Model model, Authentication authentication)
```

Purpose: Displays student's weekly class schedule

Data Processing:

1. Get student's enrolled courses
2. Get schedules for each course
3. Organize by day and time
4. Create timetable matrix

Data Structure for View:

```
Map<DayOfWeek, Map<TimeSlot, ScheduleDTO>> timetable = new HashMap<>();
```

View Rendering:

- Displays as weekly grid
- Each cell shows course code, name, classroom
- Color-coded by course

---

viewEnrollments(Model model, Authentication authentication)

```
@GetMapping("/student/enrollments")
public String viewEnrollments(Model model, Authentication authentication)
```

Purpose: Shows student's enrollment list and status

Data Categories:

- Pending enrollments (awaiting approval)
- Approved enrollments (active)
- Rejected enrollments
- Dropped enrollments

View Features:

- Status badges (color-coded)
- Enrollment date
- Action buttons (drop course if allowed)

24.2 Lecturer Dashboard Controller Methods

LecturerDashboardController Class:

showDashboard(Model model, Authentication authentication)

```
@GetMapping("/lecturer/dashboard")
public String showDashboard(Model model, Authentication authentication)
```

Purpose: Displays lecturer's main dashboard

Dashboard Statistics:

- Number of courses teaching
- Total students enrolled across courses
- Today's classes
- Pending attendance records
- Recent student activities

---

showMyCourses(Model model, Authentication authentication)

```
@GetMapping("/lecturer/courses")
public String showMyCourses(Model model, Authentication authentication)
```

Purpose: Lists all courses assigned to lecturer

For Each Course Displays:

- Course code and name
- Number of enrolled students
- Schedule information
- Quick action links

---

viewCourseDetails(Long id, Model model, Authentication auth)

```
@GetMapping("/lecturer/courses/{id}")
public String viewCourseDetails(@PathVariable Long id,
                                Model model,
                                Authentication authentication)
```

Purpose: Shows detailed view of a specific course

@PathVariable Explanation:

- URL: `/lecturer/courses/5`
- `@PathVariable Long id` extracts `5` from URL
- No need for `@RequestParam`

Data Loaded:

- Course information
- Enrolled students list
- Schedule details

- Attendance statistics
- Recent activities

Authorization Check:

- Verify lecturer owns the course
- Throw `UnauthorizedAccessException` if not

---

showAttendanceForm(Long courseId, Model model)

```
@GetMapping("/lecturer/attendance/mark")
public String showAttendanceForm(@RequestParam Long courseId,
                                 @RequestParam(required = false) LocalDate date,
                                 Model model)
```

Purpose: Displays attendance marking form

@RequestParam Explanation:

- URL: `/lecturer/attendance/mark?courseId=5&date=2026-01-17`
- `@RequestParam Long courseId` extracts `5`
- `@RequestParam(required = false) LocalDate date` extracts date (optional)

Process Flow:

1. Fetch course details
2. Get enrolled students list
3. If date provided, load existing attendance
4. Prepare form with student list
5. Render attendance marking page

Form Structure:

```
<form>
  <input type="hidden" name="courseId" value="5"/>
  <input type="date" name="date"/>
  <table>
    <tr th:each="student : ${students}">
      <td th:text="${student.name}">John Doe</td>
      <td>
        <select name="attendance_${student.id}">
          <option value="PRESENT">Present</option>
          <option value="ABSENT">Absent</option>
          <option value="LATE">Late</option>
        </select>
      </td>
    </tr>
  </table>
```

```
    <button type="submit">Save Attendance</button>
</form>
```

---

submitAttendance(AttendanceFormDTO formDTO, Authentication auth)

```
@PostMapping("/lecturer/attendance/submit")
public String submitAttendance(@ModelAttribute AttendanceFormDTO formDTO,
                               Authentication authentication,
                               RedirectAttributes redirectAttributes)
```

Purpose: Processes attendance form submission

AttendanceFormDTO Structure:

```
public class AttendanceFormDTO {
    private Long courseId;
    private LocalDate date;
    private Map<Long, String> studentAttendance; // studentId -> status
    private String notes;
}
```

Process Flow:

1. **Extract Data:**
    - Get course ID and date
    - Parse attendance map
2. **Create Attendance Records:**

```
for (Map.Entry<Long, String> entry :
formDTO.getStudentAttendance().entrySet()) {
    Long studentId = entry.getKey();
    String status = entry.getValue();

    AttendanceDTO dto = new AttendanceDTO();
    dto.setStudentId(studentId);
    dto.setCourseId(courseId);
    dto.setDate(date);
    dto.setStatus(AttendanceStatus.valueOf(status));
    dto.setMarkedBy(authentication.getName());

    attendanceService.markAttendance(dto);
}
```

3. **Sync to Google Sheets:**
    - Async call to sync service
    - Updates spreadsheet in background
4. **Response:**
    - Success message
    - Redirect to attendance view

24.3 Admin Controller Methods

AdminController Class:

showDashboard(Model model)

```
@GetMapping("/admin/dashboard")
@PreAuthorize("hasRole('ADMIN')")
public String showDashboard(Model model)
```

Purpose: Admin main dashboard with system overview

System Statistics:

```
DashboardStatsDTO stats = new DashboardStatsDTO();
stats.setTotalStudents(studentService.count());
stats.setTotalLecturers(lecturerService.count());
stats.setTotalCourses(courseService.count());
stats.setActiveEnrollments(enrollmentService.countByStatus(APPROVED));
stats.setPendingEnrollments(enrollmentService.countByStatus(PENDING));
```

Recent Activities:

- Recent enrollments
- Recent user registrations
- Recent attendance records
- System alerts

---

listUsers(String role, Model model)

```
@GetMapping("/admin/users")
public String listUsers(@RequestParam(required = false) String role,
                        Model model)
```

Purpose: User management page with filtering

Role Filter:

- If `role` is null: Show all users
- If `role = "STUDENT"`: Show only students
- If `role = "LECTURER"`: Show only lecturers
- If `role = "ADMIN"`: Show only admins

Implementation:

```
List<User> users;
if (role == null) {
    users = userService.getAllUsers();
} else {
    users = userService.getUsersByRole(role);
}
model.addAttribute("users", users);
model.addAttribute("selectedRole", role);
```

---

createUserForm(Model model)

```
@GetMapping("/admin/users/create")
public String createUserForm(Model model)
```

Purpose: Shows user creation form

Form Data:

- Empty UserDTO for form binding
- List of departments
- List of available roles

```
model.addAttribute("userDTO", new UserDTO());
model.addAttribute("departments", departmentService.getAll());
model.addAttribute("roles", Arrays.asList("STUDENT", "LECTURER", "ADMIN"));
```

---

createUser(UserDTO dto, BindingResult result)

```
@PostMapping("/admin/users/create")
public String createUser(@Valid @ModelAttribute UserDTO dto,
                         BindingResult result,
                         RedirectAttributes redirectAttributes)
```

Purpose: Processes user creation

Validation Checks:

1. **JSR-303 Validation:** @Valid triggers validation annotations

```
@NotBlank(message = "Username required")
@Size(min = 3, max = 50)
private String username;

@Email(message = "Invalid email")
private String email;
```

2. **Custom Validation:**

```
if (userService.existsByUsername(dto.getUsername())) {
    result.rejectValue("username", "error.username", "Username already
exists");
}
if (userService.existsByEmail(dto.getEmail())) {
    result.rejectValue("email", "error.email", "Email already exists");
}
```

3. **Error Handling:**

```
if (result.hasErrors()) {
    model.addAttribute("departments", departmentService.getAll());
    return "admin/user-create"; // Return to form with errors
}
```

4. **Success Path:**

```
// Create user based on role
if ("STUDENT".equals(dto.getRole())) {
    studentService.createStudent(convertToStudentDTO(dto));
}

redirectAttributes.addFlashAttribute("success", "User created
successfully!");
return "redirect:/admin/users";
```

---

approveEnrollment(Long enrollmentId, RedirectAttributes redirectAttrs)

```
@PostMapping("/admin/enrollments/{id}/approve")
public String approveEnrollment(@PathVariable Long enrollmentId,
                                RedirectAttributes redirectAttributes)
```

Purpose: Approves pending enrollment

@PathVariable vs @RequestParam:

- `@PathVariable`: Part of URL path (`/enrollments/5/approve`)
- `@RequestParam`: Query parameter (`/enrollments/approve?id=5`)

Process:

1. Call `enrollmentService.approveEnrollment(enrollmentId)`
2. Service checks course capacity
3. Updates status to APPROVED
4. Sends notification (if implemented)
5. Redirects with success message

---

rejectEnrollment(Long enrollmentId, String reason)

```
@PostMapping("/admin/enrollments/{id}/reject")
public String rejectEnrollment(@PathVariable Long enrollmentId,
                               @RequestParam(required = false) String reason,
                               RedirectAttributes redirectAttributes)
```

Purpose: Rejects enrollment with optional reason

Parameters:

- Path variable for enrollment ID
- Request param for rejection reason (optional)

24.4 REST API Controller Methods

CourseRestController Class:

getAllCourses()

```
@GetMapping("/api/courses")
public ResponseEntity<List<CourseDTO>> getAllCourses()
```

Purpose: REST API endpoint to get all courses

HTTP Response Structure:

```
{
  "status": 200,
  "body": [
    {
      "id": 1,
      "courseCode": "CS101",
      "courseName": "Introduction to Programming",
      "credits": 3,
      "maxStudents": 40,
      "currentEnrollment": 25
    }
  ]
}
```

ResponseEntity Explained:

- Wrapper for HTTP response
- Contains status code, headers, body
- `ResponseEntity.ok(data)` creates 200 OK response

---

getCourseById(Long id)

```
@GetMapping("/api/courses/{id}")
public ResponseEntity<CourseDTO> getCourseById(@PathVariable Long id)
```

Purpose: Get single course by ID

Error Handling:

```
try {
    CourseDTO course = courseService.getCourseById(id);
    return ResponseEntity.ok(course);
} catch (ResourceNotFoundException e) {
    return ResponseEntity.notFound().build(); // 404
}
```

---

createCourse(CourseDTO dto)

```
@PostMapping("/api/courses")
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<CourseDTO> createCourse(@Valid @RequestBody CourseDTO dto)
```

Purpose: Create course via REST API

@RequestBody Explanation:

- Parses JSON request body into Java object
- Uses Jackson for deserialization
- Automatic validation with @Valid

Request Example:

```
POST /api/courses
Content-Type: application/json

{
  "courseCode": "CS101",
  "courseName": "Intro to Programming",
  "credits": 3,
  "maxStudents": 40,
  "lecturerId": 5
}
```

Response:

```
CourseDTO created = courseService.createCourse(dto);
return ResponseEntity.status(HttpStatus.CREATED).body(created); // 201 Created
```

---

updateCourse(Long id, CourseDTO dto)

```
@PutMapping("/api/courses/{id}")
@PreAuthorize("hasAnyRole('ADMIN', 'LECTURER')")
public ResponseEntity<CourseDTO> updateCourse(@PathVariable Long id,
                                              @Valid @RequestBody CourseDTO dto)
```

Purpose: Update course via REST API

HTTP Method: PUT (full update) or PATCH (partial update)

Authorization:

- `@PreAuthorize("hasAnyRole('ADMIN', 'LECTURER')")`
- Allows both admins and lecturers
- SpEL (Spring Expression Language) for complex rules

---

deleteCourse(Long id)

```
@DeleteMapping("/api/courses/{id}")
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<Void> deleteCourse(@PathVariable Long id)
```

Purpose: Delete course

Response:

```
courseService.deleteCourse(id);
return ResponseEntity.noContent().build(); // 204 No Content
```

HTTP Status Codes:

- 200 OK: Success with body
- 201 Created: Resource created
- 204 No Content: Success without body
- 400 Bad Request: Validation error
- 404 Not Found: Resource not found
- 500 Internal Server Error: Server error

---

25. REPOSITORY LAYER METHODS

25.1 JPA Repository Methods

StudentRepository Interface:

Standard CRUD Methods (Inherited from JpaRepository)

```
public interface StudentRepository extends JpaRepository<Student, Long> {
    // Inherited methods (no implementation needed):
    // - save(Student) - Create or update
    // - findById(Long) - Find by primary key
    // - findAll() - Get all records
    // - deleteById(Long) - Delete by ID
    // - count() - Count total records
    // - existsById(Long) - Check existence
}
```

How JPA Generates Implementation:

- Spring Data JPA creates proxy at runtime
- No need to write implementation class

- Magic happens through naming conventions

Custom Query Methods

Method Name Query Derivation:

```
// Spring generates SQL from method name
Optional<Student> findByStudentId(String studentId);
// Generated SQL: SELECT * FROM students WHERE student_id = ?

Optional<Student> findByUsername(String username);
// Generated SQL: SELECT * FROM students WHERE username = ?

List<Student> findByMajor(String major);
// Generated SQL: SELECT * FROM students WHERE major = ?

List<Student> findByYearLevel(Integer yearLevel);
// Generated SQL: SELECT * FROM students WHERE year_level = ?

List<Student> findByDepartmentId(Long departmentId);
// Generated SQL: SELECT * FROM students WHERE department_id = ?
```

Naming Convention Rules:

- `findBy` - SELECT query
- `countBy` - COUNT query
- `deleteBy` - DELETE query
- `And` - AND condition
- `Or` - OR condition
- `OrderBy` - ORDER BY clause

Complex Examples:

```
List<Student> findByMajorAndYearLevel(String major, Integer yearLevel);
// WHERE major = ? AND year_level = ?

List<Student> findByFirstNameContaining(String name);
// WHERE first_name LIKE %?%

List<Student> findByGpaGreaterThan(Double gpa);
// WHERE gpa > ?

List<Student> findByYearLevelOrderByGpaDesc(Integer yearLevel);
// WHERE year_level = ? ORDER BY gpa DESC
```

Custom @Query Methods

JPQL Queries:

```
@Query("SELECT s FROM Student s WHERE s.isActive = true")
List<Student> findAllActiveStudents();
```

JPQL Explained:

- Java Persistence Query Language
- Queries entities, not tables
- Uses class names and property names
- Database-independent

Named Parameters:

```
@Query("SELECT s FROM Student s WHERE s.major = :major AND s.yearLevel = :year")
List<Student> findByMajorAndYear(@Param("major") String major,
                                 @Param("year") Integer year);
```

JOIN FETCH (Solve N+1 Problem):

```
@Query("SELECT DISTINCT s FROM Student s " +
       "LEFT JOIN FETCH s.enrollments e " +
       "LEFT JOIN FETCH e.course " +
       "WHERE s.id = :studentId")
Optional<Student> findByIdWithEnrollments(@Param("studentId") Long studentId);
```

N+1 Problem Explanation:

```
// BAD: N+1 queries
List<Student> students = studentRepo.findAll(); // 1 query
for (Student s : students) {
    s.getEnrollments().size(); // N queries (one per student)
}

// GOOD: Single query with JOIN FETCH
List<Student> students = studentRepo.findAllWithEnrollments(); // 1 query
for (Student s : students) {
    s.getEnrollments().size(); // No additional query
}
```

Native SQL Queries

When to Use:

- Database-specific features
- Complex queries
- Performance optimization

```
@Query(value = "SELECT * FROM students s " +
               "WHERE s.gpa > ?1 " +
               "ORDER BY s.gpa DESC " +
               "LIMIT ?2",
       nativeQuery = true)
List<Student> findTopStudentsByGpa(Double minGpa, Integer limit);
```

25.2 Enrollment Repository Methods

EnrollmentRepository Interface:

```
public interface EnrollmentRepository extends JpaRepository<Enrollment, Long> {

    // Method name derivation
    List<Enrollment> findByStudentId(Long studentId);
    List<Enrollment> findByCourseId(Long courseId);
    List<Enrollment> findByStatus(EnrollmentStatus status);
    boolean existsByStudentIdAndCourseId(Long studentId, Long courseId);

    // Custom query with JOIN FETCH
    @Query("SELECT e FROM Enrollment e " +
           "JOIN FETCH e.student s " +
           "JOIN FETCH e.course c " +
           "JOIN FETCH c.lecturer " +
           "WHERE e.student.id = :studentId")
    List<Enrollment> findByStudentIdWithCourseAndLecturer(@Param("studentId") Long
studentId);

    // Aggregate query
    @Query("SELECT COUNT(e) FROM Enrollment e " +
           "WHERE e.course.id = :courseId AND e.status = 'APPROVED'")
    Long countApprovedEnrollmentsByCourse(@Param("courseId") Long courseId);

    // Complex filtering
    @Query("SELECT e FROM Enrollment e " +
           "JOIN FETCH e.student " +
           "JOIN FETCH e.course " +
           "WHERE e.status = :status")
    List<Enrollment> findByStatusWithStudentAndCourse(@Param("status")
EnrollmentStatus status);
}
```

Method Naming Advantages:

- No SQL to write
- Type-safe
- Compile-time checking
- Auto-completion in IDE

@Query Advantages:

- Complex logic
- Performance optimization
- Explicit control

---

26. SECURITY METHODS

26.1 Authentication Methods

CustomUserDetailsService Class:

loadUserByUsername(String username)

```
@Override
public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException
```

Purpose: Core Spring Security method for authentication

Called When: User attempts to login

Process Flow:

1. **Database Lookup:**

```
User user = userRepository.findByUsername(username)
    .orElseThrow(() -> new UsernameNotFoundException("User not found: " +
username));
```

2. **Account Status Check:**

```
if (!user.getIsActive()) {
    throw new DisabledException("User account is disabled");
}
```

3. **Load Authorities (Roles):**

```
Set<GrantedAuthority> authorities = user.getRoles().stream()
    .map(role -> new SimpleGrantedAuthority(role.getName()))
    .collect(Collectors.toSet());
```

- Converts Role entities to Spring Security GrantedAuthority
- Role names like "ROLE_STUDENT" become authorities

4. **Create UserDetails Object:**

```
return new org.springframework.security.core.userdetails.User(
    user.getUsername(),              // username
    user.getPassword(),              // encoded password
    user.getIsActive(),              // enabled
    true,                            // accountNonExpired
    true,                            // credentialsNonExpired
    true,                            // accountNonLocked
    authorities                      // granted authorities
);
```

Return Value: UserDetails object that Spring Security uses for:

- Password verification
- Authority checking
- Session management

Why Important:

- Bridge between database and Spring Security
- Controls who can login
- Determines user permissions

Authentication Process Flow:

```
1. User submits login form
   ↓
2. Spring Security intercepts request
   ↓
3. Calls loadUserByUsername(username)
   ↓
4. Retrieves UserDetails from database
   ↓
5. Compares submitted password with stored password
   ↓
6. If match: Creates Authentication object
   ↓
7. Stores in SecurityContext
```

```
    ↓
8. User is authenticated
```

26.2 Password Encoding Methods

BCryptPasswordEncoder:

encode(String rawPassword)

```
String encodedPassword = passwordEncoder.encode("myPassword123");
```

Purpose: One-way hash of password

How BCrypt Works:

1. **Salt Generation:** Random salt generated for each password
2. **Hashing:** Combines password + salt, applies BCrypt algorithm
3. **Result Format:** `$2a$10$roundsAndSalt.hashedPassword`

Example:

```
Raw Password: "student123"
Encoded: "$2a$10$6YdXcN5z8ZXn3cC3w5ELUeGGP0IRZD0RvHnGGTxLi2J5c1T.Q7Py2"
```

Strength Parameter:

- `new BCryptPasswordEncoder(10)` - 10 rounds
- Higher = more secure, slower
- Range: 4-31
- Recommended: 10-12

---

matches(String rawPassword, String encodedPassword)

```
boolean isCorrect = passwordEncoder.matches(submittedPassword, storedPassword);
```

Purpose: Verify password without decoding

Process:

1. Extract salt from stored hash
2. Hash submitted password with same salt
3. Compare results
4. Return true if match

Why Can't Decode:

- BCrypt is one-way hash
- Mathematically impossible to reverse
- Must hash and compare

26.3 Authorization Methods

@PreAuthorize Annotation:

Method-Level Security

```
@PreAuthorize("hasRole('ADMIN')")
public void deleteUser(Long userId) {
    // Only accessible by ADMIN role
}
```

How It Works:

1. Spring AOP creates proxy around method
2. Before method executes, checks authorization
3. If fails, throws AccessDeniedException
4. If succeeds, proceeds to method

SpEL Expressions:

```
// Single role
@PreAuthorize("hasRole('STUDENT')")

// Multiple roles (OR)
@PreAuthorize("hasAnyRole('ADMIN', 'LECTURER')")

// Multiple roles (AND)
@PreAuthorize("hasRole('ADMIN') and hasRole('SUPER_ADMIN')")

// Check authentication
@PreAuthorize("isAuthenticated()")

// Complex expression
@PreAuthorize("hasRole('LECTURER') and #courseId ==
authentication.principal.courseId")

// Custom method call
@PreAuthorize("@securityService.canAccessCourse(authentication, #courseId)")
```

Custom Authorization Logic

SecurityService Class:

```
@Service
public class SecurityService {

    public boolean canAccessCourse(Authentication auth, Long courseId) {
        String username = auth.getName();

        // Check if lecturer owns the course
        if (auth.getAuthorities().stream()
                .anyMatch(a -> a.getAuthority().equals("ROLE_LECTURER"))) {
            Lecturer lecturer = lecturerRepo.findByUsername(username);
            return courseRepo.existsByIdAndLecturerId(courseId, lecturer.getId());
        }

        // Check if student is enrolled
        if (auth.getAuthorities().stream()
                .anyMatch(a -> a.getAuthority().equals("ROLE_STUDENT"))) {
            Student student = studentRepo.findByUsername(username);
            return enrollmentRepo.existsByStudentIdAndCourseId(student.getId(),
courseId);
        }

        return false;
    }
}
```

Usage:

```
@PreAuthorize("@securityService.canAccessCourse(authentication, #courseId)")
@GetMapping("/courses/{courseId}/details")
public String viewCourseDetails(@PathVariable Long courseId) {
    // Only accessible if user has permission for this specific course
}
```

26.4 Session Management Methods

SessionService Class:

createSession(String username, String sessionToken, HttpServletRequest request)

```
public void createSession(String username, String sessionToken, HttpServletRequest
request)
```

Purpose: Records user session in database

Session Data Stored:

- Username
- Session token (UUID)
- IP address
- User agent (browser info)
- Created timestamp
- Expiry timestamp (30 minutes)

Why Track Sessions:

- Security auditing
- Detect suspicious activity
- Force logout across devices
- Session analytics

---

invalidateSession(String sessionToken)

```
public void invalidateSession(String sessionToken)
```

Purpose: Ends user session

Called When:

- User logs out
- Session expires
- Force logout by admin

---

cleanupExpiredSessions()

```
@Scheduled(fixedRate = 300000) // Every 5 minutes
public void cleanupExpiredSessions()
```

Purpose: Automatically removes expired sessions

@Scheduled Annotation:

- Runs method periodically
- `fixedRate = 300000` - every 5 minutes (milliseconds)
- Background task, doesn't block application

Query:

```
sessionRepository.deleteByExpiresAtBefore(LocalDateTime.now());
```

26.5 OAuth2 Methods

OAuth2LoginSuccessHandler Class:

onAuthenticationSuccess(HttpServletRequest request, HttpServletResponse response, Authentication authentication)

```
@Override
public void onAuthenticationSuccess(HttpServletRequest request,
                                    HttpServletResponse response,
                                    Authentication authentication) throws
IOException
```

Purpose: Handles successful Google OAuth2 login

Process Flow:

1. **Extract OAuth2 User Data:**

```
OAuth2User oauth2User = (OAuth2User) authentication.getPrincipal();
String email = oauth2User.getAttribute("email");
String name = oauth2User.getAttribute("name");
String picture = oauth2User.getAttribute("picture");
```

2. **Check if User Exists:**

```
User user = userRepository.findByEmail(email).orElse(null);
```

3. **Create New User if Needed:**

```
if (user == null) {
    user = new User();
    user.setEmail(email);
    user.setUsername(email);
    user.setFirstName(extractFirstName(name));
    user.setLastName(extractLastName(name));
    user.setProfilePicture(picture);
    user.setPassword(""); // OAuth users don't have password
    user.setIsActive(true);

    // Assign default role
    Role studentRole = roleRepository.findByName("ROLE_STUDENT")
        .orElseThrow();
    user.setRoles(Set.of(studentRole));
```

```
        userRepository.save(user);
    }
```

4. **Redirect Based on Role:**

```
    if (hasRole(user, "ROLE_ADMIN")) {
        response.sendRedirect("/admin/dashboard");
    } else if (hasRole(user, "ROLE_LECTURER")) {
        response.sendRedirect("/lecturer/dashboard");
    } else {
        response.sendRedirect("/student/dashboard");
    }
```

OAuth2 Flow Diagram:

```
User clicks "Login with Google"
    ↓
Redirected to Google login page
    ↓
User authorizes application
    ↓
Google redirects back with authorization code
    ↓
Spring Security exchanges code for access token
    ↓
Retrieves user info from Google
    ↓
onAuthenticationSuccess() called
    ↓
User created/updated in database
    ↓
Session created
    ↓
Redirected to dashboard
```

---

27. VALIDATION METHODS

27.1 JSR-303 Bean Validation

DTO Validation Annotations:

```
public class StudentDTO {

    @NotNull(message = "ID cannot be null")
    private Long id;
```

```
    @NotBlank(message = "Student ID is required")
    @Size(min = 5, max = 20, message = "Student ID must be between 5 and 20
characters")
    @Pattern(regexp = "^STU[0-9]+$", message = "Student ID must start with STU
followed by numbers")
    private String studentId;

    @NotBlank(message = "Username is required")
    @Size(min = 3, max = 50)
    private String username;

    @NotBlank(message = "Email is required")
    @Email(message = "Invalid email format")
    private String email;

    @NotBlank(message = "First name is required")
    @Size(max = 100)
    private String firstName;

    @NotBlank(message = "Last name is required")
    @Size(max = 100)
    private String lastName;

    @Pattern(regexp = "^[0-9]{10}$", message = "Phone number must be 10 digits")
    private String phoneNumber;

    @Min(value = 1, message = "Year level must be at least 1")
    @Max(value = 4, message = "Year level cannot exceed 4")
    private Integer yearLevel;

    @DecimalMin(value = "0.00", message = "GPA cannot be negative")
    @DecimalMax(value = "4.00", message = "GPA cannot exceed 4.00")
    @Digits(integer = 1, fraction = 2, message = "GPA format: X.XX")
    private Double gpa;
}
```

Validation Annotation Meanings:

- `@NotNull` - Value cannot be null
- `@NotBlank` - String cannot be null, empty, or whitespace
- `@NotEmpty` - Collection/Array cannot be null or empty
- `@Size(min, max)` - String/Collection size constraints
- `@Min(value)` - Numeric minimum value
- `@Max(value)` - Numeric maximum value
- `@Email` - Valid email format
- `@Pattern(regexp)` - Must match regex pattern
- `@DecimalMin/Max` - Decimal number constraints
- `@Digits(integer, fraction)` - Number digit constraints
- `@Past` - Date must be in past
- `@Future` - Date must be in future

Controller Validation Trigger

```
@PostMapping("/students/create")
public String createStudent(@Valid @ModelAttribute StudentDTO dto,
                            BindingResult result,
                            Model model) {

    // Check for validation errors
    if (result.hasErrors()) {
        // Errors are automatically bound to BindingResult
        model.addAttribute("departments", departmentService.getAll());
        return "student/create"; // Return to form with error messages
    }

    // Validation passed, proceed
    studentService.createStudent(dto);
    return "redirect:/admin/students";
}
```

How @Valid Works:

1. Spring binds form data to DTO
2. @Valid triggers validation
3. Violations added to BindingResult
4. Check `result.hasErrors()`
5. Display errors in view if any

Displaying Errors in Thymeleaf:

```
<form th:object="${studentDTO}">
    <div class="form-group">
        <label>Student ID</label>
        <input type="text" th:field="*{studentId}" class="form-control"
               th:classappend="${#fields.hasErrors('studentId')} ? 'is-invalid' :
''"/>
        <div class="invalid-feedback" th:if="${#fields.hasErrors('studentId')}"
             th:errors="*{studentId}">
            Error message
        </div>
    </div>
</form>
```

27.2 Custom Validator Classes

CourseValidator Class:

validateCourseCreation(CourseDTO dto)

```
public void validateCourseCreation(CourseDTO dto)
```

Purpose: Business logic validation beyond basic constraints

Validation Checks:

1. **Course Code Uniqueness:**

```
if (courseRepository.existsByCourseCode(dto.getCourseCode())) {
    throw new DuplicateResourceException("Course code already exists");
}
```

2. **Credits Range:**

```
if (dto.getCredits() <= 0 || dto.getCredits() > 6) {
    throw new IllegalArgumentException("Credits must be between 1 and 6");
}
```

3. **Max Students Positive:**

```
if (dto.getMaxStudents() <= 0) {
    throw new IllegalArgumentException("Max students must be positive");
}
```

4. **Lecturer Availability:**

```
if (dto.getLecturerId() != null) {
    Lecturer lecturer = lecturerRepo.findById(dto.getLecturerId())
        .orElseThrow(() -> new ResourceNotFoundException("Lecturer not
found"));

    // Check if lecturer is overloaded
    long courseCount = courseRepo.countByLecturerId(dto.getLecturerId());
    if (courseCount >= 5) {
        throw new IllegalStateException("Lecturer already teaching maximum
courses");
    }
}
```

---

validateEnrollment(Long studentId, Long courseId)

```
public void validateEnrollment(Long studentId, Long courseId)
```

Purpose: Validates enrollment business rules

Validation Checks:

1. **Course Exists and Active:**

```
Course course = courseRepository.findById(courseId)
    .orElseThrow(() -> new ResourceNotFoundException("Course not found"));

if (course.getStatus() != CourseStatus.ACTIVE) {
    throw new IllegalStateException("Course is not active for enrollment");
}
```

2. **Course Capacity:**

```
long enrolledCount =
enrollmentRepository.countApprovedEnrollmentsByCourse(courseId);
if (enrolledCount >= course.getMaxStudents()) {
    throw new IllegalStateException("Course is full");
}
```

3. **No Duplicate Enrollment:**

```
if (enrollmentRepository.existsByStudentIdAndCourseId(studentId, courseId))
{
    throw new DuplicateResourceException("Already enrolled in this course");
}
```

4. **Prerequisites Check:**

```
if (course.hasPrerequisites()) {
    List<Course> prerequisites = course.getPrerequisites();
    for (Course prereq : prerequisites) {
        boolean completed =
enrollmentRepository.existsByStudentIdAndCourseIdAndStatusAndGradeGreaterTha
n(
            studentId, prereq.getId(), EnrollmentStatus.COMPLETED, "C"
        );
        if (!completed) {
            throw new IllegalStateException("Prerequisite not met: " +
```

```
prereq.getCourseCode());
        }
    }
}
```

5. **Credit Limit Check:**

```
int currentCredits = enrollmentRepository.sumCreditsByStudentIdAndSemester(
    studentId, currentSemester.getId()
);
if (currentCredits + course.getCredits() > 24) {
    throw new IllegalStateException("Enrollment would exceed credit limit
(24 credits)");
}
```

27.3 Schedule Conflict Detection

ConflictDetectionService Class:

hasScheduleConflict(ScheduleDTO dto)

```
public boolean hasScheduleConflict(ScheduleDTO dto)
```

Purpose: Detects scheduling conflicts before saving

Conflict Types Checked:

1. **Classroom Conflict:**

```
// Same classroom, same day, overlapping time
List<Schedule> classroomSchedules =
scheduleRepository.findByClassroomIdAndDayOfWeek(
    dto.getClassroomId(), dto.getDayOfWeek()
);

for (Schedule existing : classroomSchedules) {
    // Skip if comparing with self (during update)
    if (dto.getId() != null && dto.getId().equals(existing.getId())) {
        continue;
    }

    if (isTimeOverlapping(dto.getStartTime(), dto.getEndTime(),
                          existing.getStartTime(), existing.getEndTime())) {
        return true; // Conflict found
    }
}
```

2. **Lecturer Conflict:**

```
// Same lecturer, same day, overlapping time
Course course = courseRepository.findById(dto.getCourseId()).orElse(null);
if (course != null && course.getLecturer() != null) {
    List<Schedule> lecturerSchedules =
scheduleRepository.findByLecturerIdAndDayOfWeek(
        course.getLecturer().getId(), dto.getDayOfWeek()
    );

    for (Schedule existing : lecturerSchedules) {
        if (dto.getId() != null && dto.getId().equals(existing.getId())) {
            continue;
        }

        if (isTimeOverlapping(dto.getStartTime(), dto.getEndTime(),
                              existing.getStartTime(),
existing.getEndTime())) {
            return true; // Conflict found
        }
    }
}
```

3. **Student Conflict (Optional):**

```
// Check if enrolled students have conflicting schedules
List<Student> enrolledStudents = enrollmentRepository
    .findStudentsByCourseId(dto.getCourseId());

for (Student student : enrolledStudents) {
    List<Schedule> studentSchedules = scheduleRepository
        .findByStudentIdAndDayOfWeek(student.getId(), dto.getDayOfWeek());

    for (Schedule existing : studentSchedules) {
        if (isTimeOverlapping(dto.getStartTime(), dto.getEndTime(),
                              existing.getStartTime(),
existing.getEndTime())) {
            return true; // Student has conflict
        }
    }
}
```

---

isTimeOverlapping(LocalTime start1, LocalTime end1, LocalTime start2, LocalTime end2)

```
private boolean isTimeOverlapping(LocalTime start1, LocalTime end1,
                                  LocalTime start2, LocalTime end2)
```

Purpose: Determines if two time ranges overlap

Logic:

```
// Two time ranges overlap if:
// start1 < end2 AND end1 > start2

return start1.isBefore(end2) && end1.isAfter(start2);
```

Examples:

```
Case 1: Overlap
Schedule 1: 09:00 - 10:30
Schedule 2: 10:00 - 11:30
Result: true (overlaps 10:00-10:30)

Case 2: No Overlap
Schedule 1: 09:00 - 10:30
Schedule 2: 10:30 - 12:00
Result: false (back-to-back, no overlap)

Case 3: Complete Overlap
Schedule 1: 09:00 - 12:00
Schedule 2: 10:00 - 11:00
Result: true (schedule 2 completely inside schedule 1)
```

---

28. UTILITY METHODS

28.1 Date and Time Utilities

DateTimeUtils Class:

formatDate(LocalDate date)

```
public static String formatDate(LocalDate date)
```

Purpose: Formats date for display

Example:

```
LocalDate date = LocalDate.of(2026, 1, 17);
String formatted = DateTimeUtils.formatDate(date);
// Result: "January 17, 2026"
```

---

formatTime(LocalTime time)

```
public static String formatTime(LocalTime time)
```

Purpose: Formats time for display

Example:

```
LocalTime time = LocalTime.of(14, 30);
String formatted = DateTimeUtils.formatTime(time);
// Result: "2:30 PM"
```

---

calculateDuration(LocalTime start, LocalTime end)

```
public static long calculateDuration(LocalTime start, LocalTime end)
```

Purpose: Calculates duration in minutes

Implementation:

```
public static long calculateDuration(LocalTime start, LocalTime end) {
    return Duration.between(start, end).toMinutes();
}
```

Example:

```
LocalTime start = LocalTime.of(9, 0);   // 9:00 AM
LocalTime end = LocalTime.of(10, 30);    // 10:30 AM
long minutes = DateTimeUtils.calculateDuration(start, end);
// Result: 90 minutes
```

---

isWithinDateRange(LocalDate date, LocalDate start, LocalDate end)

```
public static boolean isWithinDateRange(LocalDate date, LocalDate start, LocalDate
end)
```

Purpose: Checks if date falls within range

Implementation:

```
public static boolean isWithinDateRange(LocalDate date, LocalDate start, LocalDate
end) {
    return !date.isBefore(start) && !date.isAfter(end);
}
```

28.2 String Utilities

Helper Methods in Service Classes:

generateStudentId()

```
public String generateStudentId()
```

Purpose: Generates unique student ID

Implementation:

```
public String generateStudentId() {
    String prefix = "STU";
    LocalDate now = LocalDate.now();
    String year = String.valueOf(now.getYear()).substring(2); // Last 2 digits

    // Get last student ID for this year
    String lastId = studentRepository.findLastStudentIdForYear(year);

    int nextNumber = 1;
    if (lastId != null) {
        // Extract number from last ID (e.g., "STU26001" -> "001")
        String numberPart = lastId.substring(5);
        nextNumber = Integer.parseInt(numberPart) + 1;
    }

    // Format: STU26001, STU26002, etc.
    return String.format("%s%s%03d", prefix, year, nextNumber);
}
```

Example Output:

- STU26001
- STU26002
- STU27001 (next year)

---

sanitizeInput(String input)

```
public String sanitizeInput(String input)
```

Purpose: Removes potentially harmful characters

Implementation:

```
public String sanitizeInput(String input) {
    if (input == null) return null;

    // Remove HTML tags
    String sanitized = input.replaceAll("<[^>]*>", "");

    // Remove special characters (keep alphanumeric, space, common punctuation)
    sanitized = sanitized.replaceAll("[^a-zA-Z0-9\\s.,!?-]", "");

    // Trim whitespace
    sanitized = sanitized.trim();

    return sanitized;
}
```

28.3 File Storage Utilities

FileStorageService Class:

storeFile(MultipartFile file)

```
public String storeFile(MultipartFile file)
```

Purpose: Saves uploaded file to server

Process Flow:

1. **Validate Filename:**

```
String fileName = StringUtils.cleanPath(file.getOriginalFilename());
```

```
if (fileName.contains("..")) {
    throw new IllegalArgumentException("Invalid file path: " + fileName);
}
```

2. **Generate Unique Filename:**

```
String fileExtension = getFileExtension(fileName);
String uniqueFileName = UUID.randomUUID().toString() + fileExtension;
```

   - UUID ensures uniqueness
   - Prevents filename conflicts
   - Preserves file extension

3. **Save File:**

```
Path uploadPath = Paths.get("uploads/profiles");
Files.createDirectories(uploadPath); // Create if not exists

Path targetLocation = uploadPath.resolve(uniqueFileName);
Files.copy(file.getInputStream(), targetLocation,
StandardCopyOption.REPLACE_EXISTING);
```

4. **Return Filename:**

```
return uniqueFileName; // Store in database
```

Why UUID:

- Guaranteed uniqueness
- Prevents naming conflicts
- Security (hard to guess filenames)

---

loadFileAsResource(String fileName)

```
public Resource loadFileAsResource(String fileName)
```

Purpose: Retrieves uploaded file for download/display

Implementation:

```
public Resource loadFileAsResource(String fileName) {
    try {
        Path filePath =
Paths.get("uploads/profiles").resolve(fileName).normalize();
        Resource resource = new UrlResource(filePath.toUri());

        if (resource.exists()) {
            return resource;
        } else {
            throw new ResourceNotFoundException("File not found: " + fileName);
        }
    } catch (MalformedURLException ex) {
        throw new ResourceNotFoundException("File not found: " + fileName);
    }
}
```

Usage in Controller:

```
@GetMapping("/files/{filename:.+}")
public ResponseEntity<Resource> downloadFile(@PathVariable String filename) {
    Resource resource = fileStorageService.loadFileAsResource(filename);

    return ResponseEntity.ok()
        .header(HttpHeaders.CONTENT_DISPOSITION,
                "attachment; filename=\"" + resource.getFilename() + "\"")
        .body(resource);
}
```

---

deleteFile(String fileName)

```
public void deleteFile(String fileName)
```

Purpose: Removes file from server

Implementation:

```
public void deleteFile(String fileName) {
    try {
        Path filePath =
Paths.get("uploads/profiles").resolve(fileName).normalize();
        Files.deleteIfExists(filePath);
    } catch (IOException ex) {
        throw new RuntimeException("Could not delete file: " + fileName);
    }
}
```

When Called:

- User deletes profile picture
- User uploads new picture (delete old one)
- Account deletion

28.4 Data Conversion Utilities

DTO Converter Pattern:

Entity to DTO Conversion

```
public StudentDTO convertToDTO(Student entity)
```

Purpose: Safely expose entity data

Why Convert:

1. **Security:** Hide sensitive fields (password)
2. **Performance:** Reduce data transfer size
3. **Flexibility:** Customize response format
4. **Versioning:** API changes don't affect database

Example:

```
public StudentDTO convertToDTO(Student student) {
    StudentDTO dto = new StudentDTO();
    dto.setId(student.getId());
    dto.setStudentId(student.getStudentId());
    dto.setFirstName(student.getFirstName());
    dto.setLastName(student.getLastName());
    dto.setEmail(student.getEmail());
    // Note: password NOT included

    // Add computed fields
    dto.setFullName(student.getFirstName() + " " + student.getLastName());

    if (student.getDepartment() != null) {
        dto.setDepartmentName(student.getDepartment().getDepartmentName());
    }

    return dto;
}
```

DTO to Entity Conversion

```
public Student convertToEntity(StudentDTO dto)
```

Purpose: Create entity from DTO for persistence

Example:

```
public Student convertToEntity(StudentDTO dto) {
    Student student = new Student();
    student.setStudentId(dto.getStudentId());
    student.setUsername(dto.getUsername());
    student.setEmail(dto.getEmail());
    student.setFirstName(dto.getFirstName());
    student.setLastName(dto.getLastName());

    // Encode password
    student.setPassword(passwordEncoder.encode(dto.getPassword()));

    // Fetch and set relationships
    if (dto.getDepartmentId() != null) {
        Department dept = departmentRepo.findById(dto.getDepartmentId())
            .orElseThrow(() -> new ResourceNotFoundException("Department not
found"));
        student.setDepartment(dept);
    }

    return student;
}
```

---

29. GOOGLE SHEETS INTEGRATION METHODS

29.1 Google Sheets Service Methods

GoogleSheetsServiceImpl Class:

createCourseSheet(Long courseId, String courseName)

```
public String createCourseSheet(Long courseId, String courseName) throws
IOException
```

Purpose: Creates new Google Sheet for course attendance tracking

Process Flow:

1. **Create Spreadsheet:**

```
Spreadsheet spreadsheet = new Spreadsheet()
    .setProperties(new SpreadsheetProperties()
        .setTitle(courseName + " - Attendance"));

spreadsheet = sheetsService.spreadsheets().create(spreadsheet).execute();
String spreadsheetId = spreadsheet.getSpreadsheetId();
```

2. **Set Up Headers:**

```
List<List<Object>> headers = Arrays.asList(
    Arrays.asList("Date", "Student ID", "Student Name", "Status", "Notes")
);

ValueRange headerRange = new ValueRange().setValues(headers);

sheetsService.spreadsheets().values()
    .update(spreadsheetId, "Sheet1!A1:E1", headerRange)
    .setValueInputOption("RAW")
    .execute();
```

3. **Format Headers:**

```
List<Request> requests = new ArrayList<>();
requests.add(new Request()
    .setRepeatCell(new RepeatCellRequest()
        .setRange(new GridRange()
            .setSheetId(0)
            .setStartRowIndex(0)
            .setEndRowIndex(1))
        .setCell(new CellData()
            .setUserEnteredFormat(new CellFormat()
                .setBackgroundColor(new Color()
                    .setRed(0.2f)
                    .setGreen(0.6f)
                    .setBlue(0.86f))
                .setTextFormat(new TextFormat()
                    .setBold(true)
                    .setForegroundColor(new Color()
                        .setRed(1f)
                        .setGreen(1f)
                        .setBlue(1f)))))
        .setFields("userEnteredFormat(backgroundColor,textFormat)")));

BatchUpdateSpreadsheetRequest batchRequest =
    new BatchUpdateSpreadsheetRequest().setRequests(requests);

sheetsService.spreadsheets().batchUpdate(spreadsheetId,
batchRequest).execute();
```

4. **Save to Database:**

```
CourseSheet courseSheet = new CourseSheet();
courseSheet.setCourseId(courseId);
courseSheet.setSpreadsheetId(spreadsheetId);
courseSheet.setSheetName("Sheet1");
courseSheet.setCreatedAt(LocalDateTime.now());
courseSheetRepository.save(courseSheet);
```

5. **Return Spreadsheet ID:**

```
return spreadsheetId;
```

Result: Creates formatted Google Sheet with headers and links it to course

---

syncAttendanceToSheet(Long courseId, List attendanceList)

```
public void syncAttendanceToSheet(Long courseId, List<Attendance> attendanceList)
throws IOException
```

Purpose: Syncs attendance records to Google Sheets

Process Flow:

1. **Get Course Sheet:**

```
CourseSheet courseSheet = courseSheetRepository.findByCourseId(courseId)
    .orElseThrow(() -> new ResourceNotFoundException("Course sheet not
found"));
```

2. **Prepare Data:**

```
List<List<Object>> values = new ArrayList<>();
for (Attendance attendance : attendanceList) {
    values.add(Arrays.asList(
        attendance.getAttendanceDate().toString(),
        attendance.getStudent().getStudentId(),
        attendance.getStudent().getFirstName() + " " +
            attendance.getStudent().getLastName(),
        attendance.getStatus().toString(),
        attendance.getNotes() != null ? attendance.getNotes() : ""
```

```
        ));
    }
```

3. **Append to Sheet:**

```
ValueRange body = new ValueRange().setValues(values);

sheetsService.spreadsheets().values()
    .append(courseSheet.getSpreadsheetId(),
           courseSheet.getSheetName() + "!A2",
           body)
    .setValueInputOption("RAW")
    .setInsertDataOption("INSERT_ROWS")
    .execute();
```

4. **Update Sync Status:**

```
attendanceList.forEach(att -> {
    att.setGoogleSheetSynced(true);
    att.setLastSyncedAt(LocalDateTime.now());
});
attendanceRepository.saveAll(attendanceList);
```

API Methods Used:

- `spreadsheets().values().append()` - Adds new rows
- `setValueInputOption("RAW")` - No formatting, raw values
- `setInsertDataOption("INSERT_ROWS")` - Insert new rows instead of overwriting

---

readAttendanceFromSheet(Long courseId)

```
public List<AttendanceDTO> readAttendanceFromSheet(Long courseId) throws
IOException
```

Purpose: Reads attendance data from Google Sheet

Process Flow:

1. **Get Course Sheet:**

```
CourseSheet courseSheet = courseSheetRepository.findByCourseId(courseId)
    .orElseThrow(() -> new ResourceNotFoundException("Course sheet not
found"));
```

2. **Read Data:**

```
ValueRange response = sheetsService.spreadsheets().values()
    .get(courseSheet.getSpreadsheetId(),
         courseSheet.getSheetName() + "!A2:E")
    .execute();

List<List<Object>> values = response.getValues();
```

3. **Parse and Convert:**

```
List<AttendanceDTO> attendanceList = new ArrayList<>();

if (values != null && !values.isEmpty()) {
    for (List<Object> row : values) {
        AttendanceDTO dto = new AttendanceDTO();
        dto.setAttendanceDate(LocalDate.parse(row.get(0).toString()));
        dto.setStudentId(row.get(1).toString());
        dto.setStudentName(row.get(2).toString());
        dto.setStatus(row.get(3).toString());
        dto.setNotes(row.size() > 4 ? row.get(4).toString() : "");
        attendanceList.add(dto);
    }
}
```

4. **Return Data:**

```
return attendanceList;
```

Range Notation:

- `"A2:E"` - Columns A through E, starting from row 2
- `"A2:E"` - Read until last row with data
- `"A2:E100"` - Explicit end at row 100

---

updateSheetCell(String spreadsheetId, String range, Object value)

```
public void updateSheetCell(String spreadsheetId, String range, Object value)
throws IOException
```

Purpose: Updates single cell or range

Example Usage:

```
// Update single cell
updateSheetCell(sheetId, "A2", "PRESENT");

// Update row
updateSheetCell(sheetId, "A2:E2",
    Arrays.asList("2026-01-17", "STU001", "John Doe", "PRESENT", ""));
```

Implementation:

```
public void updateSheetCell(String spreadsheetId, String range, Object value)
        throws IOException {
    List<List<Object>> values;
    if (value instanceof List) {
        values = Arrays.asList((List<Object>) value);
    } else {
        values = Arrays.asList(Arrays.asList(value));
    }

    ValueRange body = new ValueRange().setValues(values);

    sheetsService.spreadsheets().values()
        .update(spreadsheetId, range, body)
        .setValueInputOption("RAW")
        .execute();
}
```

---

deleteSheetRows(String spreadsheetId, int startRow, int endRow)

```
public void deleteSheetRows(String spreadsheetId, int startRow, int endRow) throws
IOException
```

Purpose: Removes rows from sheet

Implementation:

```
public void deleteSheetRows(String spreadsheetId, int startRow, int endRow)
        throws IOException {
    DeleteDimensionRequest deleteRequest = new DeleteDimensionRequest()
        .setRange(new DimensionRange()
            .setSheetId(0)
            .setDimension("ROWS")
            .setStartIndex(startRow)
            .setEndIndex(endRow));

    Request request = new Request().setDeleteDimension(deleteRequest);
```

```
    BatchUpdateSpreadsheetRequest batchRequest =
        new BatchUpdateSpreadsheetRequest()
            .setRequests(Arrays.asList(request));

    sheetsService.spreadsheets().batchUpdate(spreadsheetId,
batchRequest).execute();
}
```

29.2 Google Sheets API Configuration

sheetsService() Bean

```
@Bean
public Sheets sheetsService() throws IOException, GeneralSecurityException
```

Purpose: Creates authenticated Google Sheets service

Process Flow:

1. **Load Credentials:**

```
InputStream credentialsStream = new FileInputStream(credentialsFile);

GoogleCredentials credentials = GoogleCredentials
    .fromStream(credentialsStream)
    .createScoped(Collections.singleton(SheetsScopes.SPREADSHEETS));
```

2. **Create HTTP Transport:**

```
HttpRequestInitializer requestInitializer =
    new HttpCredentialsAdapter(credentials);
```

3. **Build Sheets Service:**

```
return new Sheets.Builder(
    GoogleNetHttpTransport.newTrustedTransport(),
    GsonFactory.getDefaultInstance(),
    requestInitializer)
    .setApplicationName(applicationName)
    .build();
```

Configuration Properties:

```
google.credentials.file=src/main/resources/credentials/service-account.json
google.application.name=University Course Enrollment System
```

---

30. TRANSACTION MANAGEMENT METHODS

30.1 @Transactional Annotation

Purpose: Ensures database operations are atomic (all-or-nothing)

How It Works:

```
@Transactional
public void enrollStudentInCourse(Long studentId, Long courseId) {
    // Step 1: Create enrollment
    Enrollment enrollment = new Enrollment();
    enrollment.setStudentId(studentId);
    enrollment.setCourseId(courseId);
    enrollmentRepository.save(enrollment);

    // Step 2: Update course count
    courseService.updateEnrollmentCount(courseId);

    // Step 3: Send notification
    notificationService.sendEnrollmentNotification(studentId, courseId);

    // If ANY step fails, ALL steps are rolled back
}
```

Without @Transactional:

- Step 1 succeeds, saves enrollment
- Step 2 fails with exception
- Step 3 never executes
- **Problem:** Enrollment created but count not updated (inconsistent state)

With @Transactional:

- Step 1 executes (not committed yet)
- Step 2 fails with exception
- **Transaction rolls back:** Step 1 is undone
- Database remains consistent

30.2 Transaction Attributes

Read-Only Transactions

```
@Transactional(readOnly = true)
public List<Student> getAllStudents() {
    return studentRepository.findAll();
}
```

Benefits:

- Performance optimization
- Prevents accidental modifications
- Database can optimize read operations

Isolation Levels

```
@Transactional(isolation = Isolation.READ_COMMITTED)
public void updateStudentGPA(Long studentId, Double gpa) {
    Student student = studentRepository.findById(studentId).orElseThrow();
    student.setGpa(gpa);
    studentRepository.save(student);
}
```

Isolation Levels:

- `READ_UNCOMMITTED` - Can read uncommitted changes (dirty reads)
- `READ_COMMITTED` - Only read committed data (default)
- `REPEATABLE_READ` - Same read twice returns same result
- `SERIALIZABLE` - Full isolation, slowest

Propagation

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void logActivity(String action) {
    // Always creates new transaction
    // Even if called within another transaction
    activityRepository.save(new Activity(action));
}
```

Propagation Types:

- `REQUIRED` - Join existing or create new (default)
- `REQUIRES_NEW` - Always create new transaction
- `NESTED` - Create nested transaction
- `MANDATORY` - Must be called within transaction
- `NEVER` - Must not be called within transaction

Rollback Rules

```
@Transactional(rollbackFor = Exception.class)
public void createUser(UserDTO dto) {
    // Rolls back on any exception
    userRepository.save(convertToEntity(dto));
}

@Transactional(noRollbackFor = CustomException.class)
public void updateUser(Long id, UserDTO dto) {
    // Does NOT rollback on CustomException
    userRepository.save(convertToEntity(dto));
}
```

Default Rollback Behavior:

- **Runtime Exceptions:** Triggers rollback
- **Checked Exceptions:** Does NOT rollback
- Configure with `rollbackFor` and `noRollbackFor`

30.3 Programmatic Transaction Management

Using TransactionTemplate:

```
@Service
public class PaymentService {

    private final TransactionTemplate transactionTemplate;

    public PaymentService(PlatformTransactionManager transactionManager) {
        this.transactionTemplate = new TransactionTemplate(transactionManager);
    }

    public void processPayment(PaymentDTO payment) {
        transactionTemplate.execute(status -> {
            try {
                // Transactional operations
                paymentRepository.save(payment);
                accountService.deductBalance(payment.getAmount());
                ledgerService.recordTransaction(payment);

                return null; // Success
            } catch (Exception e) {
                status.setRollbackOnly(); // Mark for rollback
                throw e;
            }
        });
    }
}
```

31. ASYNC AND SCHEDULED METHODS

31.1 Asynchronous Methods

@Async Annotation:

```
@Service
public class EmailService {

    @Async
    public CompletableFuture<Void> sendEnrollmentConfirmation(String email, String
courseName) {
        // This runs in separate thread
        // Does not block calling code

        try {
            MimeMessage message = mailSender.createMimeMessage();
            message.setTo(email);
            message.setSubject("Enrollment Confirmation");
            message.setText("You are enrolled in " + courseName);

            mailSender.send(message);

            return CompletableFuture.completedFuture(null);
        } catch (Exception e) {
            return CompletableFuture.failedFuture(e);
        }
    }
}
```

Usage:

```
// Non-blocking call
emailService.sendEnrollmentConfirmation("student@example.com", "CS101");
// Code continues immediately without waiting for email to send
```

Configuration:

```
@Configuration
@EnableAsync
public class AsyncConfig {

    @Bean
    public Executor taskExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
```

```
        executor.setCorePoolSize(5);
        executor.setMaxPoolSize(10);
        executor.setQueueCapacity(100);
        executor.setThreadNamePrefix("async-");
        executor.initialize();
        return executor;
    }
}
```

31.2 Scheduled Methods

@Scheduled Annotation:

Fixed Rate

```
@Scheduled(fixedRate = 60000) // Every 60 seconds
public void syncAttendanceToGoogleSheets() {
    List<Attendance> unsyncedRecords =
attendanceRepository.findByGoogleSheetSyncedFalse();

    for (Attendance attendance : unsyncedRecords) {
        try {
            googleSheetsService.syncAttendance(attendance);
            attendance.setGoogleSheetSynced(true);
            attendanceRepository.save(attendance);
        } catch (Exception e) {
            log.error("Failed to sync attendance: " + attendance.getId(), e);
        }
    }
}
```

Runs: Every 60 seconds, regardless of how long method takes

Fixed Delay

```
@Scheduled(fixedDelay = 300000) // 5 minutes after previous execution completes
public void cleanupExpiredSessions() {
    sessionRepository.deleteByExpiresAtBefore(LocalDateTime.now());
}
```

Runs: 5 minutes AFTER previous execution completes

Cron Expression

```
@Scheduled(cron = "0 0 2 * * ?") // Every day at 2:00 AM
public void generateDailyReports() {
    // Generate attendance reports
    // Send to administrators
    // Clean up old logs
}

@Scheduled(cron = "0 0 12 * * MON-FRI") // Every weekday at noon
public void sendReminderEmails() {
    // Send attendance reminders to lecturers
}
```

Cron Format: `second minute hour day month weekday`

- `0 0 2 * * ?` - 2:00 AM every day
- `0 */15 * * * ?` - Every 15 minutes
- `0 0 0 1 * ?` - 1st day of every month at midnight

Configuration

```
@Configuration
@EnableScheduling
public class SchedulingConfig {
    // Enables @Scheduled annotation processing
}
```

---

32. METHOD NAMING CONVENTIONS SUMMARY

32.1 Service Layer Method Names

CRUD Operations:

```
create{Entity}(DTO dto)           // Create new entity
update{Entity}(Long id, DTO dto)  // Update existing entity
delete{Entity}(Long id)           // Delete entity
get{Entity}ById(Long id)          // Retrieve by ID
getAll{Entities}()                // Retrieve all
```

Querying:

```
find{Entity}By{Criteria}()        // Find entities by criteria
get{Entity}sBy{Criteria}()        // Alternative to find
search{Entities}(SearchDTO dto)   // Complex search
```

Business Logic:

```
approve{Action}(Long id)          // Approve operation
reject{Action}(Long id)           // Reject operation
process{Operation}(DTO dto)       // Process business operation
calculate{Result}(params)         // Calculation methods
validate{Entity}(DTO dto)         // Validation methods
```

32.2 Controller Method Names

Web Controllers:

```
show{Page}()                      // GET - Display page
edit{Entity}()                    // GET - Show edit form
create{Entity}Form()              // GET - Show create form
```

Form Processing:

```
save{Entity}()                    // POST - Save entity
update{Entity}()                  // POST - Update entity
delete{Entity}()                  // POST - Delete entity
```

REST Controllers:

```
get{Entity}()                     // GET - Retrieve
create{Entity}()                  // POST - Create
update{Entity}()                  // PUT/PATCH - Update
delete{Entity}()                  // DELETE - Delete
```

32.3 Repository Method Names

Spring Data JPA Conventions:

```
findBy{Property}                  // WHERE property = ?
findBy{Property}And{Property2}    // WHERE property = ? AND property2 = ?
findBy{Property}Or{Property2}     // WHERE property = ? OR property2 = ?
findBy{Property}OrderBy{Prop}Desc // ORDER BY
countBy{Property}                 // COUNT WHERE
deleteBy{Property}                // DELETE WHERE
existsBy{Property}                // Boolean check
```

CONCLUSION

This comprehensive explanation covers all major methods and functions used throughout the University Course Enrollment System. Each method serves a specific purpose in the application architecture:

1. **Service Layer**: Business logic and data processing
2. **Controller Layer**: Request handling and response generation
3. **Repository Layer**: Database access and queries
4. **Security Layer**: Authentication and authorization
5. **Validation Layer**: Data integrity and business rules
6. **Utility Layer**: Helper functions and common operations
7. **Integration Layer**: External service connections

Understanding these methods provides insight into:

- How data flows through the application
- How business rules are enforced
- How security is implemented
- How external services are integrated
- How transactions maintain data consistency
